

Defensa completa SOS2526-29

Documento canonico del proyecto para la defensa. Es la fuente principal para estudiar, modificar y explicar el proyecto; el `README.md` queda como entrada breve al repositorio.

Sustituye toda la documentacion de defensa que antes estaba repartida en varias guias. Esas guias antiguas ya no se conservan como fuentes independientes dentro de `docs/`.

Objetivo: que una persona que no conoce el repositorio pueda entender que hace el proyecto, como se instala, como se ejecuta, como esta organizado, que rutas existen, que funciones importan, como fluyen los datos y como defenderlo ante un profesor o tribunal.

Nota de nombre: el archivo real del repositorio es `docs/DEFENSA_COMPLETA.md`. Cuando en conversaciones o planes aparezca `DEFENSA_COMPLETA.md`, se refiere a este documento.

Fuente actualizada para estudiar y modificar: este Markdown. Si se quiere una copia PDF para imprimir, debe regenerarse desde esta version.

Indice

Seccion	Contenido	Paginas
	1. Resumen ejecutivo	p. 3
	2. Problema, objetivos y publico	p. 3
	3. Equipo y recursos	p. 4
	4. Stack tecnologico	p. 5
	5. Arquitectura general	p. 6
	6. Estructura real de carpetas	p. 7
	7. Instalacion, configuracion y ejecucion	pp. 8-9
	8. Scripts, tests y validaciones	p. 10
	9. Rutas utiles para abrir	p. 11
	10. Modelo de datos	p. 12
	11. Backend: arranque y responsabilidades	pp. 12-13
	12. APIs REST del proyecto	p. 14
	13. Frontend: rutas, servicios y pantallas	pp. 15-16
	14. Parte LCC: citys-stats	pp. 17-19
	15. Integraciones LCC	pp. 20-22
	16. Funciones importantes explicadas	pp. 23-49
	17. Flujos de ejecucion	pp. 50-63
	18. Codigos HTTP y contrato REST	pp. 64-66
	19. Cambios que pueden pedir en directo	pp. 67-71
	20. Errores comunes y soluciones	pp. 72-73
	21. Decisiones tecnicas	p. 74
	22. Limitaciones y mejoras futuras	p. 75
	23. Guion de defensa practica	pp. 76-94
	24. Preguntas dificiles y respuestas	pp. 95-96
	25. Glosario para personas no tecnicas	p. 97
	26. Checklists finales	pp. 97-98
	27. Anexos tecnicos	p. 99

Ruta de lectura recomendada para defender LCC `citys-stats`:

1. Primero lee las secciones 1 a 5 para entender el proyecto comun y poder abrir la defensa sin perderte.
2. Despues lee las secciones 10 a 15 para centrarte en el modelo, API, frontend e integraciones de `citys-stats`.
3. Estudia muy bien la seccion 16: ahi esta el codigo de funciones, como explicarlas y como modificarlas.
4. Usa las secciones 17 a 20 para responder flujos, codigos HTTP, cambios en directo y errores.
5. Termina con las secciones 23, 24 y 26 para practicar la defensa practica, preguntas dificiles y checklist final.

1. Resumen ejecutivo

`SOS2526-29` es una aplicacion web de la asignatura SOS2526. Tiene backend y frontend y permite gestionar, consultar, visualizar e integrar tres recursos de datos:

- `natural-disasters` : desastres naturales por pais y anio.
- `citys-stats` : poblacion estimada de ciudades para 2025.
- `wine-stats` : informacion de vinos, precios, tipos y caracteristicas.

La aplicacion permite:

- Listar datos.
- Buscar y filtrar datos.
- Crear registros.
- Editar registros en pantallas separadas.
- Borrar registros concretos.
- Borrar colecciones completas.
- Cargar datos iniciales.
- Mostrar graficas con Highcharts.
- Mostrar mapas.
- Consumir integraciones externas mediante proxy propio.

Frase corta de defensa:

! Nuestro proyecto es una aplicacion web con Node.js, Express, NeDB y Svelte. El backend ofrece una API REST para tres recursos: `natural-disasters` , `citys-stats` y `wine-stats` . El frontend consume esas APIs con `fetch` , muestra CRUDs, graficas, mapas e integraciones. Los datos se guardan en archivos NeDB y las APIs externas se consumen desde el backend para controlar errores y normalizar las respuestas.

2. Problema, objetivos y publico

Problema que resuelve

El proyecto organiza varias fuentes de datos heterogeneas en una aplicacion unica. En vez de consultar JSON crudo o APIs separadas, el usuario puede navegar por pantallas, ver tablas, cargar datos, hacer operaciones CRUD y entender los datos mediante visualizaciones.

Objetivos funcionales

- Exponer una API REST para cada recurso.
- Mantener versiones de API cuando un recurso evoluciona.
- Persistir datos localmente con NeDB.
- Ofrecer una interfaz web usable con Svelte.
- Validar entradas para evitar datos incompletos o inconsistentes.
- Probar el contrato de API con Newman.
- Probar flujos de usuario con Playwright.
- Mostrar visualizaciones individuales y grupales.
- Integrar APIs externas sin que el navegador dependa directamente de ellas.

Objetivos de defensa

- Saber explicar el arranque completo desde `npm start`.
- Saber localizar cada capa: servidor, API, servicio frontend y pantalla.
- Saber justificar REST, versionado, codigos HTTP, validaciones y proxy.
- Saber hacer cambios pequenos en directo sin perderse.
- Saber responder por que `citys-stats` usa v2 para CRUD y v1 para integraciones.

Publico objetivo

- Profesor o evaluador de SOS2526.
- Companeros del grupo.
- Cualquier persona tecnica que quiera ejecutar o revisar el proyecto.
- Cualquier stakeholder no tecnico que quiera entender el resultado final.

3. Equipo y recursos

Miembro	Recurso	API principal	Frontend
Rufino Moreno Pacheco	<code>wine-stats</code>	<code>/api/v1/wine-stats</code>	<code>/wine-stats</code>
Luis Cortes Cobos	<code>citys-stats</code>	<code>/api/v2/citys-stats</code>	<code>/citys-stats</code>
Alberto Lirola Gomez	<code>natural-disasters</code>	<code>/api/v2/natural-disasters</code>	<code>/natural-disasters</code>

Repositorio:

<https://github.com/gti-sos/SOS2526-29>

Aplicacion desplegada:

<https://sos2526-29.onrender.com/>

4. Stack tecnologico

Tecnologia	Donde aparece	Para que se usa
Node.js	<code>index.js</code> , scripts npm	Ejecutar el backend
Express	<code>index.js</code> , <code>src/back/**</code>	Definir servidor y rutas REST
<code>@seald-io/nedb</code>	<code>index.js</code> , <code>src/back/*.db</code>	Persistencia local en archivos
CORS	<code>index.js</code>	Permitir llamadas desde Vite en desarrollo
Svelte	<code>frontend-group/src</code>	Construir la interfaz
Vite	<code>frontend-group/vite.config.js</code>	Desarrollo y build del frontend
Highcharts	analytics e integraciones	Graficas, mapas y widgets
<code>@highcharts/map-collection</code>	mapa de ciudades	TopoJSON del mapa mundial
Newman	<code>tests/*.json</code>	Ejecutar colecciones Postman
Playwright	<code>tests*/e2e</code>	Pruebas end-to-end
<code>start-server-and-test</code>	<code>package.json</code>	Arrancar servidor y lanzar tests

Dependencias principales en `package.json` :

```
express
@seald-io/nedb
cors
svelte
highcharts
@highcharts/map-collection
leaflet
cool-ascii-faces
```

Dependencias del frontend en `frontend-group/package.json` :

```
@sveltejs/vite-plugin-svelte
vite
svelte
svelte-spa-router
highcharts
@highcharts/map-collection
```

Nota: aunque aparece `svelte-spa-router` , el enrutado actual no usa rutas `#/` : `App.svelte` descubre pantallas con `import.meta.glob` y resuelve `window.location.pathname` ; `navigation.js` usa `history.pushState` para navegar dentro de la SPA.

5. Arquitectura general

El proyecto sigue una arquitectura por capas:

```

Navegador
|
| HTML, CSS, JS compilado
v
Frontend Svelte en public/
|
| fetch HTTP JSON
v
Backend Express en index.js
|
| registra modulos REST
v
src/back/v1 y src/back/v2
|
| consultas y modificaciones
v
Bases NeDB en src/back/*.db

Integraciones externas:

Frontend Svelte -> API propia /api/v1/citys-stats/integrations/... -> fetch desde Express -> APIs externas JSON
  
```

Separacion principal:

- `index.js` : arranque, middleware, bases de datos, registro de APIs, frontend estatico y fallback SPA.
- `src/back/v1` y `src/back/v2` : contratos REST por recurso y version.
- `frontend-group/src/services` : funciones `fetch` que encapsulan URLs y errores.
- `frontend-group/src/routes` : pantallas visibles, organizadas por carpetas con `+page.svelte`.
- `public` : build final del frontend que sirve Express en produccion.
- `tests` : validaciones API y navegador.

Frase de defensa:

- ! La aplicacion separa responsabilidades: Svelte solo pinta y llama a servicios, Express valida y coordina, NeDB persiste, y las integraciones externas pasan por nuestro backend para evitar problemas de origen y controlar errores.

6. Estructura real de carpetas

Estructura resumida:

```
SOS2526-29/
|-- index.js
|-- package.json
|-- package-lock.json
|-- README.md
|-- docs/
|   |-- DEFENSA_COMPLETA.md
|   |-- DEFENSA_COMPLETA.pdf
|-- src/
|   |-- back/
|   |   |-- v1/
|   |   |   |-- citys-stats.js
|   |   |   |-- natural-disasters.js
|   |   |   |-- wine-stats.js
|   |   |-- v2/
|   |   |   |-- citys-stats.js
|   |   |   |-- natural-disasters.js
|   |   |-- citys-stats.db
|   |   |-- natural-disasters.db
|   |   |-- wine-stats.db
|-- frontend-group/
|   |-- package.json
|   |-- vite.config.js
|   |-- src/
|   |   |-- App.svelte
|   |   |-- main.js
|   |   |-- app.css
|   |   |-- components/
|   |   |   |-- Navbar.svelte
|   |   |-- lib/
|   |   |   |-- navigation.js
|   |   |-- services/
|   |   |   |-- apiBase.js
|   |   |   |-- citysStatsApi.js
|   |   |   |-- citysStatsIntegrations.js
|   |   |   |-- natural-disasters.js
|   |   |   |-- wine-stats.js
|   |   |-- routes/
|   |   |   |-- +page.svelte
|   |   |   |-- citys-stats/
|   |   |   |-- natural-disasters/
|   |   |   |-- wine-stats/
|   |   |   |-- analytics/
|   |   |   |-- integrations/
|-- public/
|-- tests/
|   |-- ALG/
|   |-- LCC/
|   |-- RMP/
|-- playwright.config.js
```

Archivos clave:

Archivo	Importancia
<code>index.js</code>	Punto de entrada del backend y servidor unico
<code>src/back/v2/citys-stats.js</code>	API principal del CRUD LCC
<code>src/back/v1/citys-stats.js</code>	API v1 LCC, agregados por pais e integraciones
<code>frontend-group/src/App.svelte</code>	Router real de la SPA
<code>frontend-group/src/components/Navbar.svelte</code>	Menu superior
<code>frontend-group/src/routes/+page.svelte</code>	Portada
<code>frontend-group/src/routes/citys-stats/+page.svelte</code>	CRUD LCC
<code>frontend-group/src/routes/citys-stats/editar/[city]/[country]/+page.svelte</code>	Edicion LCC
<code>frontend-group/src/routes/analytics/+page.svelte</code>	Widget grupal
<code>frontend-group/src/routes/analytics/citys-stats/+page.svelte</code>	Grafico individual LCC
<code>frontend-group/src/routes/analytics/citys-stats/map/+page.svelte</code>	Mapa LCC
<code>frontend-group/src/routes/integrations/citys-stats/+page.svelte</code>	Integraciones LCC
<code>frontend-group/src/services/citysStatsApi.js</code>	Fetch CRUD LCC v2
<code>frontend-group/src/services/citysStatsIntegrations.js</code>	Fetch integraciones LCC v1
<code>frontend-group/vite.config.js</code>	Build hacia <code>../public</code>

Rutas canonicas LCC:

- La ruta canonica para defensa es `citys-stats`, porque ese es el nombre del recurso publicado.
- Las rutas antiguas `city-stats` se eliminaron para evitar duplicidad en el frontend.

7. Instalacion, configuracion y ejecucion

Requisitos

- Node.js instalado.
- npm instalado.
- Acceso a internet si se van a probar integraciones externas.

Instalacion desde cero

En la raiz del repositorio:

```
npm.cmd install
npm.cmd --prefix frontend-group install
```

Tambien funciona:

```
npm install
npm --prefix frontend-group install
```

En PowerShell se recomienda `npm.cmd` si aparece el error de `npm.ps1`.

Build del frontend

```
npm.cmd run build
```

Que hace:

1. Entra en `frontend-group`.
2. Ejecuta `vite build`.
3. Genera el frontend compilado en `public`.
4. Borra previamente `public` por `emptyOutDir: true`.

Arranque local

```
npm.cmd start
```

Equivale a:

```
node index.js
```

URL local:

```
http://localhost:10000
```

El puerto por defecto esta en `index.js`:

```
const port = process.env.PORT || 10000;
```

En Render manda `process.env.PORT`; en local, si no se define, usa `10000`.

Desarrollo frontend con Vite

```
npm.cmd run dev-front
```

Ese script ejecuta:

```
cd frontend-group && npm run dev -- --open
```

Durante desarrollo, `frontend-group/vite.config.js` tiene proxy:

```
/api -> http://localhost:10000
```

Ademas, `apiBase.js` fuerza las llamadas API a `http://localhost:10000` cuando `import.meta.env.DEV` es verdadero.

Variables de entorno reconocidas

Variable	Uso
PORT	Puerto del servidor Express
LCC_DOCS_URL	URL de documentacion Postman v1 de citys-stats
LCC_DOCS_V2_URL	URL de documentacion Postman v2 de citys-stats

8. Scripts, tests y validaciones

Scripts principales de package.json :

Script	Comando real	Uso
npm start	node index.js	Arrancar servidor
npm run build	cd frontend-group && npm run build	Compilar Svelte a public
npm run dev-front	cd frontend-group && npm run dev -- --open	Desarrollo frontend
npm run test-citys-stats	Newman LCC v1 local	API ciudades v1
npm run test-citys-stats-v2	Newman LCC v2 local	API ciudades v2
npm run test-LCC	Arranca servidor y ejecuta Newman LCC v1	Validacion LCC v1
npm run test-LCC-v2	Arranca servidor y ejecuta Newman LCC v2	Validacion LCC v2
npm run test-LCC-e2e	Playwright con tests/LCC/playwright.config.js	Flujo navegador LCC
npm test	Suite por defecto del repositorio	Validacion completa del grupo

Comandos recomendados antes de defender:

```
npm.cmd run build
npm.cmd start
```

En otra terminal, si se quiere validar:

```
npm.cmd run test-LCC-v2
npm.cmd run test-LCC-e2e
```

Tests LCC e2e cubren:

- Portada con tarjeta de Luis y enlaces LCC.
- Listado de citys-stats .
- Creacion de ciudad.
- Borrado individual.
- Borrado total.
- Edicion en /citys-stats/editar/:city/:country .
- Busqueda con filtros, orden y limite.

9. Rutas utiles para abrir

Local

```
http://localhost:10000/
http://localhost:10000/citys-stats
http://localhost:10000/analytics
http://localhost:10000/analytics/citys-stats
http://localhost:10000/analytics/citys-stats/map
http://localhost:10000/integrations
http://localhost:10000/integrations/citys-stats
http://localhost:10000/api/v2/citys-stats
http://localhost:10000/api/v1/citys-stats
```

Desplegado

```
https://sos2526-29.onrender.com/
https://sos2526-29.onrender.com/citys-stats
https://sos2526-29.onrender.com/analytics
https://sos2526-29.onrender.com/analytics/citys-stats
https://sos2526-29.onrender.com/analytics/citys-stats/map
https://sos2526-29.onrender.com/integrations
https://sos2526-29.onrender.com/integrations/citys-stats
https://sos2526-29.onrender.com/api/v2/citys-stats/docs
```

Orden recomendado para la defensa LCC:

Orden	Ruta	Que demostrar
1	/	Portada, miembros, enlaces de APIs y documentacion
2	/api/v2/citys-stats/docs	Documentacion Postman de API v2
3	/citys-stats	CRUD completo, filtros, ordenacion y paginacion
4	/analytics/citys-stats	Highcharts individual no lineal
5	/analytics/citys-stats/map	Mapa geoespacial
6	/integrations	Entrada comun de integraciones
7	/integrations/citys-stats	7 integraciones por pais con proxy
8	/analytics	Widget grupal unico

Comprobaciones rapidas con PowerShell:

```
Invoke-WebRequest http://localhost:10000/ -UseBasicParsing
Invoke-WebRequest http://localhost:10000/citys-stats -UseBasicParsing
Invoke-WebRequest http://localhost:10000/analytics -UseBasicParsing
Invoke-WebRequest http://localhost:10000/integrations/citys-stats -UseBasicParsing
Invoke-WebRequest "http://localhost:10000/api/v1/citys-stats/integrations/summary?limit=8" -UseBasicParsing
```

10. Modelo de datos

citys-stats

Ejemplo:

```
{
  "city": "tokyo",
  "country": "japan",
  "un_2025_population": 33412512
}
```

Clave del recurso:

city + country

Campos:

Campo	Tipo esperado	Significado
city	texto	Ciudad
country	texto	País
un_2025_population	entero positivo	Poblacion estimada por Naciones Unidas para 2025

Datos iniciales principales:

jakarta, dhaka, tokyo, delhi, shanghai, guangzhou, cairo, manila, kolkata, seoul, karachi, mumbai

11. Backend: arranque y responsabilidades

Archivo principal:

index.js

Responsabilidades:

1. Importa Express, path, NeDB y CORS.
2. Crea `app`.
3. Define `port`.
4. Activa `cors()`.
5. Activa `express.json()` para leer cuerpos JSON.
6. Crea tres bases NeDB:
 - `naturalDisastersDb`
 - `citysStatsDb`
 - `wineStatsDb`
1. Carga y registra modulos de API:

- `src/back/v1/natural-disasters.js`
- `src/back/v2/natural-disasters.js`
- `src/back/v1/citys-stats.js`
- `src/back/v2/citys-stats.js`
- `src/back/v1/wine-stats.js`

1. Sirve el frontend compilado desde `public`.
2. Define `/` y `/about` para devolver el frontend compilado. La ruta `/about` se renderiza en Svelte desde `frontend-group/src/routes/about/+page.svelte`.
3. Define proxy `/api/proxy/exportations-stats`.
4. Define fallback para toda ruta que no empiece por `/api`.
5. Lanza `app.listen`.

Frase de defensa:

! `index.js` es el punto donde se juntan las capas. Crea el servidor, abre las bases, registra las APIs REST y sirve la SPA de Svelte desde `public`.

Bases de datos

Variable	Archivo
<code>naturalDisastersDb</code>	<code>src/back/natural-disasters.db</code>
<code>citysStatsDb</code>	<code>src/back/citys-stats.db</code>
<code>wineStatsDb</code>	<code>src/back/wine-stats.db</code>

Todas usan:

```
autoload: true
```

Eso abre el archivo al arrancar.

Fallback de la SPA

Esta ruta es clave:

```
app.get(/^\/(?!api\/).*/, (request, response) => {  
  response.sendFile(frontendIndexPath);  
});
```

Significa:

- Si la ruta no empieza por `/api`, se devuelve `public/index.html`.
- Svelte decide despues que pantalla mostrar.
- Por eso funcionan rutas directas como `/analytics` o `/citys-stats/editar/tokyo/japan`.

12. APIs REST del proyecto

12.1 citys-stats v2

Archivo:

```
src/back/v2/citys-stats.js
```

Ruta base:

```
/api/v2/citys-stats
```

Uso principal:

- CRUD de LCC desde el frontend.
- Búsqueda libre.
- Filtros exactos.
- Ordenacion.
- Paginacion.
- Validacion estricta.

Endpoints:

Metodo	Ruta	Resultado
GET	/api/v2/citys-stats/docs	Redirige a Postman
GET	/api/v2/citys-stats/loadInitialData	Inserta datos si la base esta vacia o devuelve existentes
GET	/api/v2/citys-stats	Lista con filtros, <code>q</code> , <code>sort</code> , <code>offset</code> , <code>limit</code>
GET	/api/v2/citys-stats/:city/:country	Obtiene un registro
POST	/api/v2/citys-stats	Crea un registro
POST	/api/v2/citys-stats/:city/:country	405
PUT	/api/v2/citys-stats	405
PUT	/api/v2/citys-stats/:city/:country	Actualiza registro
DELETE	/api/v2/citys-stats	Borra todos y devuelve 204
DELETE	/api/v2/citys-stats/:city/:country	Borra uno y devuelve 204

Queries utiles:

```
GET /api/v2/citys-stats?q=india
GET /api/v2/citys-stats?city=tokyo
GET /api/v2/citys-stats?country=china
GET /api/v2/citys-stats?un_2025_population=33412512
GET /api/v2/citys-stats?sort=-un_2025_population
GET /api/v2/citys-stats?sort=city&offset=0&limit=5
```

12.2 citys-stats v1

Archivo:

```
src/back/v1/citys-stats.js
```

Ruta base:

```
/api/v1/citys-stats
```

Uso principal:

- Compatibilidad v1.
- CRUD basico con filtros exactos y paginacion.
- Endpoints de agregacion.
- Proxies e integraciones externas.

Endpoints especiales:

Metodo	Ruta	Uso
GET	/api/v1/citys-stats/top-cities	Top por poblacion
GET	/api/v1/citys-stats/country-summaries	Agregado local por pais
GET	/api/v1/citys-stats/integrations/geocoding/:city	Proxy Open-Meteo
GET	/api/v1/citys-stats/integrations/country/:country	Proxy REST Countries
GET	/api/v1/citys-stats/integrations/world-bank/:countryCode	Proxy World Bank
GET	/api/v1/citys-stats/integrations/sos-tourist-arrivals	Proxy SOS2526-25
GET	/api/v1/citys-stats/integrations/sos-earthquakes	Proxy SOS2526-19
GET	/api/v1/citys-stats/integrations/sos-fifa-squad-values	Proxy SOS2526-26
GET	/api/v1/citys-stats/integrations/sos-esports-earnings	Proxy SOS2526-30
GET	/api/v1/citys-stats/integrations/summary	Resumen integrado

13. Frontend: rutas, servicios y pantallas

Punto de entrada

Archivo:

```
frontend-group/src/main.js
```

Hace:

1. Importa `mount` de Svelte.
2. Importa `app.css`.
3. Importa `App.svelte`.
4. Monta la app en `document.getElementById("app")`.

Router real

Archivo:

```
frontend-group/src/App.svelte
```

Funcionamiento:

1. Usa `import.meta.glob("./routes/**/*.svelte", { eager: true })`.
2. Convierte cada ruta de archivo en ruta web.
3. Compila segmentos dinamicos como `[city]` o `[country]` a expresiones regulares.
4. Lee `window.location.pathname`.
5. Elige el componente correspondiente.
6. Escucha `popstate` para actualizar la pantalla.

Ejemplos:

Archivo	Ruta
<code>routes/+page.svelte</code>	<code>/</code>
<code>routes/citys-stats/+page.svelte</code>	<code>/citys-stats</code>
<code>routes/citys-stats/editar/[city]/[country]/+page.svelte</code>	<code>/citys-stats/editar/:city/:country</code>
<code>routes/analytics/+page.svelte</code>	<code>/analytics</code>
<code>routes/integrations/citys-stats/+page.svelte</code>	<code>/integrations/citys-stats</code>

Navegacion interna

Archivo:

```
frontend-group/src/lib/navigation.js
```

Funciones:

- `navigate(path)`: usa `history.pushState` y dispara `PopStateEvent`.
- `replace(path)`: reemplaza la URL actual y dispara `PopStateEvent`.
- `back()`: llama a `window.history.back()`.

Services

Los services evitan duplicar URLs y parsing de errores en cada pantalla.

Service	API que llama	Uso
<code>apiBase.js</code>	Calcula origen	Local con Vite o despliegue
<code>citysStatsApi.js</code>	<code>/api/v2/citys-stats</code>	CRUD LCC
<code>citysStatsIntegrations.js</code>	<code>/api/v1/citys-stats</code>	Integraciones LCC

Pantallas principales

Ruta	Archivo
/	frontend-group/src/routes/+page.svelte
/citys-stats	frontend-group/src/routes/citys-stats/+page.svelte
/citys-stats/editar/:city/:country	frontend-group/src/routes/citys-stats/editar/[city]/[country]/+page.svelte
/analytics	frontend-group/src/routes/analytics/+page.svelte
/analytics/citys-stats	frontend-group/src/routes/analytics/citys-stats/+page.svelte
/analytics/citys-stats/map	frontend-group/src/routes/analytics/citys-stats/map/+page.svelte
/integrations	frontend-group/src/routes/integrations/+page.svelte
/integrations/citys-stats	frontend-group/src/routes/integrations/citys-stats/+page.svelte

14. Parte LCC: citys-stats

Que es

`citys-stats` gestiona estadísticas de población estimada para ciudades en 2025.

Campos:

```
city
country
un_2025_population
```

Identificador:

```
city + country
```

Por que se llama `citys-stats`

Aunque en inglés correcto sería `cities-stats`, el recurso publicado en la práctica es `citys-stats`. Cambiarlo rompería:

- URLs.
- Tests.
- Documentación Postman.
- Enlaces del frontend.
- Integraciones ya entregadas.

Frase de defensa:

! Mantengo `citys-stats` porque es el contrato público del proyecto. En APIs, estabilidad del contrato pesa más que corregir un nombre histórico.

API LCC principal

Para CRUD se usa:

```
/api/v2/citys-stats
```

Motivo:

- Tiene busqueda libre `q`.
- Tiene ordenacion `sort`.
- Tiene paginacion `limit` y `offset`.
- Tiene validacion estricta.
- Devuelve JSON limpio sin `_id`.

API LCC de integraciones

Para integraciones se usa:

```
/api/v1/citys-stats/integrations/...
```

Motivo:

- El proxy externo esta implementado en v1.
- Mantiene compatibilidad con entregas previas.
- Centraliza llamadas a Open-Meteo, REST Countries, World Bank y APIs SOS externas.

CRUD visible

Pantalla:

```
frontend-group/src/routes/citys-stats/+page.svelte
```

Funciones principales de la pantalla:

- `emptycreateForm`
- `emptySearchForm`
- `clearFeedback`
- `hasQueryValues`
- `parsePositiveInteger`
- `parseOptionalNonNegativeInteger`
- `parseOptionalPositiveInteger`
- `validateCityStatForm`
- `buildSearchQuery`
- `refreshList`
- `handleSearch`
- `handleResetSearch`
- `handleCreate`
- `handleLoadInitialData`
- `handleDeleteAll`
- `handleDeleteOne`
- `openEdit`

Edicion separada

Pantalla:

```
frontend-group/src/routes/citys-stats/editar/[city]/[country]/+page.svelte
```

Comportamiento importante:

- Carga el registro con `getOneCityStat(params.city, params.country)`.
- Si el usuario cambia solo poblacion, hace `PUT`.
- Si el usuario cambia `city` o `country`, crea el registro nuevo y borra el antiguo.
- Usa `replace(...)` para actualizar la URL cuando cambia la clave.

Esto evita una limitacion natural de `PUT`: la URL identifica el recurso original, pero si cambia la clave compuesta, en realidad nace otra URL.

Visualizacion individual

Pantalla:

```
frontend-group/src/routes/analytics/citys-stats/+page.svelte
```

Usa:

- Highcharts.
- Grafico `pie`.
- Datos de `getAllCityStats({ sort: "-un_2025_population" })`.

Frase:

! La visualizacion individual no es `line`; es un `pie` que representa la proporcion de poblacion estimada por ciudad.

Mapa

Pantalla:

```
frontend-group/src/routes/analytics/citys-stats/map/+page.svelte
```

Usa:

- Highcharts Maps.
- `@highcharts/map-collection/custom/world.topo.json`.
- Coordenadas locales en el objeto `coordinates`.
- Marcadores `mappoint`.
- Color y radio segun poblacion.

Claves de coordenadas:

```
city|country
```

Ejemplo:

```
"tokyo|japan": { lat: 35.6762, lon: 139.6503 }
```

15. Integraciones LCC

Pantalla:

```
frontend-group/src/routes/integrations/citys-stats/+page.svelte
```

Backend:

```
src/back/v1/citys-stats.js
```

Service:

```
frontend-group/src/services/citysStatsIntegrations.js
```

Decision clave: integrar por país

La integración se hace por `country`, no por ciudad.

Motivos:

- REST Countries trabaja por país.
- World Bank trabaja por país o código ISO.
- Muchas APIs SOS externas publican país o ISO3.
- Los nombres de ciudad no coinciden bien entre fuentes.
- El cruce por país produce menos huecos y más información real.

Frase de defensa:

! Integro por país porque es el campo común más estable entre mi recurso y las APIs externas. Así evito una integración frágil por nombres de ciudades y consigo widgets con datos reales comparables.

APIs integradas

API	Tipo	Fuente	Endpoint proxy local	Widget
Open-Meteo Geocoding	No SOS	<code>https://geocoding-api.open-meteo.com/v1/search</code>	<code>/api/v1/citys-stats/integrations/geocoding/:city</code>	treemap
REST Countries	No SOS	<code>https://restcountries.com/v3.1/name/:country</code>	<code>/api/v1/citys-stats/integrations/country/:country</code>	sankey
World Bank Indicators	No SOS	<code>https://api.worldbank.org/v2/country/:code/indicator/SP.POP.TOTL</code>	<code>/api/v1/citys-stats/integrations/world-bank/:countryCode</code>	lollipop
SOS2526-25 <code>international-tourist-arrivals</code>	Alumno SOS	<code>https://sos2526-25.onrender.com/api/v2/international-tourist-arrivals</code>	<code>/api/v1/citys-stats/integrations/sos-tourist-arrivals</code>	bar
SOS2526-19 <code>earthquakes</code>	Alumno SOS	<code>https://sos2526-19.onrender.com/api/v1/earthquakes</code>	<code>/api/v1/citys-stats/integrations/sos-earthquakes</code>	bullet
SOS2526-26 <code>fifa-squad-value-per-years</code>	Alumno SOS	<code>https://sos2526-26.onrender.com/api/v2/fifa-squad-value-per-years</code>	<code>/api/v1/citys-stats/integrations/sos-fifa-squad-values</code>	columnpyramid
SOS2526-30 <code>esportsearnings-stats</code>	Alumno SOS	<code>https://sos2526-30.onrender.com/api/v1/esportsearnings-stats</code>	<code>/api/v1/citys-stats/integrations/sos-esports-earnings</code>	sunburst

Cumplimiento D03

- Usa 7 APIs externas.
- Usa 3 APIs no SOS: Open-Meteo, REST Countries y World Bank.
- Usa 4 APIs SOS de otros grupos: 25, 19, 26 y 30.
- Todas devuelven JSON.
- Todas pasan por proxy propio en Express.
- No se muestra JSON crudo: se transforma en graficas, tarjetas, tablas y metricas.
- Los widgets no son `line`.
- La vista esta enlazada desde `/integrations`.
- El widget grupal esta en `/analytics`.

Flujo de integraciones

1. El usuario abre `/integrations/citys-stats`.
2. La pantalla llama a `getCityStatsIntegrationSummary(selectedLimit)`.
3. El service hace una sola peticion a `GET /api/v1/citys-stats/integrations/summary?limit=N`.
4. El backend lee NeDB y agrupa ciudades por pais con `buildCityCountrySummaries`.
5. El backend consulta Open-Meteo, REST Countries, World Bank y APIs SOS mediante proxy propio.
6. `fetchJson` cachea solo respuestas de APIs externas durante unos minutos y reutiliza peticiones externas identicas que llegan a la vez.
7. El endpoint `summary` no se cachea: cada llamada vuelve a leer los datos locales de NeDB, por lo que un alta o edicion en Render se refleja al instante en la integracion.
8. El backend normaliza datos y devuelve un resumen ya preparado.
9. El frontend transforma ese resumen en `geocodingRows`, `countryCards`, `worldBankRows` y rankings externos.
10. El frontend construye widgets Highcharts distintos sin volver a llamar a cada proxy individual.

Endpoint resumen

Ruta:

```
GET /api/v1/citys-stats/integrations/summary?limit=8
```

Respuesta real resumida:

```
{
  localResource: "/api/v1/citys-stats/country-summaries",
  externalApis: string[],
  studentApis: object[],
  studentApiDatasets: object,
  count: number,
  items: object[]
}
```

Campos importantes de cada item:

Campo	Significado
<code>city</code>	Ciudad principal del país
<code>country</code>	País usado como clave de cruce
<code>cityCount</code>	Número de ciudades locales de ese país
<code>topCity</code>	Ciudad local más poblada
<code>topCityPopulation</code>	Población de la ciudad local más poblada
<code>cities</code>	Lista de ciudades locales del país
<code>un_2025_population</code>	Suma local de población 2025 del país
<code>geocoding</code>	Datos de Open-Meteo
<code>countryInfo</code>	Datos de REST Countries
<code>worldBankPopulation</code>	Dato de población World Bank
<code>touristArrivals</code>	Datos agregados de turismo
<code>earthquakeStats</code>	Datos agregados de terremotos
<code>fifaSquadValue</code>	Datos agregados FIFA
<code>esportsEarnings</code>	Datos agregados eSports
<code>integrationErrors</code>	Errores parciales

Incidencia resuelta: demasiadas peticiones

Problema detectado:

- Un consumidor externo avisó de que, a veces, al cargar una gráfica conjunta con `citys-stats`, aparecía un error de servidor con demasiadas peticiones y la gráfica no cargaba.
- La causa probable era que la vista de integraciones hacía muchas llamadas encadenadas o paralelas desde el navegador: primero datos locales, después Open-Meteo, REST Countries, World Bank y varias APIs SOS.
- Si varios usuarios o integraciones externas cargaban la página a la vez, nuestro servidor y las APIs externas recibían demasiadas peticiones repetidas.

Solución aplicada:

- La pantalla `/integrations/citys-stats` ya no coordina todas las llamadas externas una a una desde el navegador.
- Ahora llama una sola vez a `/api/v1/citys-stats/integrations/summary?limit=N`.
- El backend monta el resumen completo, normaliza los datos y devuelve también `studentApiDatasets` para que el frontend pueda pintar todos los widgets sin llamar de nuevo a cada proxy individual.
- `fetchJson` cachea respuestas de APIs externas y colapsa peticiones idénticas simultáneas. Esto reduce carga sobre servicios externos y evita picos.
- No se cachea el resumen final ni los datos locales de `citys-stats`: si se crea, edita o borra un registro en Render, el siguiente resumen lee NeDB de nuevo y refleja el cambio inmediatamente.

Que requisitos NO cambian:

- La API principal de CRUD sigue siendo `/api/v2/citys-stats`.
- Los endpoints proxy individuales siguen existiendo bajo `/api/v1/citys-stats/integrations/...`.
- Se siguen usando APIs REST JSON.
- Sigue habiendo más de 5 integraciones, al menos 3 no SOS y al menos 2 SOS de otros grupos.
- Los datos siguen mostrándose en HTML y widgets, no como JSON crudo.
- No se cambia el tipo de gráfica individual ni el contrato del recurso `citys-stats`.

Respuesta corta de defensa:

- ! Detectamos que la integracion podia generar demasiadas peticiones repetidas. Lo solucionamos moviendo la coordinacion al backend: el frontend hace una sola peticion al endpoint `summary`, el backend consulta y normaliza las fuentes externas, cachea solo respuestas externas y devuelve los datos listos para pintar. Los datos locales no se cachean, asi que los cambios del CRUD se ven al instante.

Incidencia resuelta: pais inexistente y World Bank ISO3

Problema detectado:

- Si el usuario creaba un registro con un pais inventado, por ejemplo `wakanda`, el CRUD local podia guardarlo.
- Despues, al abrir `/integrations/citys-stats`, la integracion intentaba cruzar ese pais con REST Countries y World Bank.
- REST Countries no podia obtener un codigo ISO3 real y World Bank no podia consultar poblacion.
- El resultado visible era un aviso como `World Bank (1): Country ISO3 code not available`.

Solucion aplicada:

- En backend v1 y v2 se anadio una lista de paises admitidos: `SUPPORTED_COUNTRIES`.
- Se anadio `COUNTRY_ALIASES` para aceptar nombres habituales como `espana`, `usa`, `uk` o `corea-del-sur`.
- `normalizeCountryForStorage` convierte el pais a formato canonico: minusculas, sin tildes, con guiones y alias resueltos.
- `parseCityStat` valida estructura, poblacion y pais. Si el pais no esta soportado, devuelve `400` con `{ "error": "Invalid country" }`.
- El frontend usa `supportedCountries.js` para mostrar opciones validas y bloquear el envio antes de llamar a la API.
- `citysStatsApi.js` traduce `Invalid country` a un mensaje comprensible para el usuario.

Que se consigue:

- No se guardan paises imposibles de cruzar con APIs externas.
- Las integraciones dejan de fallar por datos locales inventados.
- El backend sigue siendo la barrera real aunque alguien se salte la interfaz y use Postman.

Respuesta corta de defensa:

- ! Antes se podia crear un pais que nuestra API local aceptaba, pero que las integraciones externas no podian resolver. Lo he corregido validando el pais en backend y frontend. Si el pais no esta en la lista soportada, la API responde `400 Invalid country` y no guarda el dato. Asi evito errores posteriores de REST Countries y World Bank.

Prueba que puedes hacer en Postman:

```
POST /api/v2/citys-stats
Body: { "city": "demo", "country": "wakanda", "un_2025_population": 1000 }
Respuesta: 400
JSON: { "error": "Invalid country" }
```

16. Funciones importantes explicadas

Esta seccion es la guia tecnica principal para defender y modificar la parte LCC `citys-stats`. La idea no es memorizar todo el repositorio, sino saber leer una funcion, reconocer en que capa esta, explicar por que existe y saber que tocar si en defensa piden un cambio.

Orden recomendado para estudiar esta seccion:

1. Leer el mapa por capas para saber donde vive cada cosa.
2. Aprender las fichas de backend v2, porque son el contrato REST principal.
3. Aprender services y pantallas, porque explican como llega la accion del usuario al backend.
4. Repasar integraciones, mapa y analytics, porque ahi suelen preguntar por asincronia y APIs externas.
5. Usar las recetas de cambio como chuleta para modificar el programa en directo.

16.1 Como defender cualquier funcion

Si el profesor senala una funcion, contesta siempre con este orden:

1. Archivo y capa: backend, service, pantalla, grafica o integracion.
2. Momento de ejecucion: al arrancar, al abrir pantalla, al pulsar boton o al recibir una peticion.
3. Entrada: parametros, req.query, req.params, req.body, estado Svelte o respuesta externa.
4. Trabajo: valida, transforma, consulta base, llama fetch, pinta grafica o actualiza estado.
5. Salida: JSON, codigo HTTP, valor devuelto, Error lanzado o variables de pantalla actualizadas.
6. Cambio probable: que linea tocaria si me piden cambiar el comportamiento.

Frase comodin segura:

- ! Esta funcion tiene una responsabilidad concreta dentro de su capa. Recibe datos, los valida o transforma, llama a la siguiente capa si hace falta y devuelve un resultado controlado. Si se cambia, hay que revisar las funciones que la llaman y los tests asociados.

16.2 Mapa de funciones LCC por capa

Capa	Archivo	Funciones o bloques que debes reconocer
Arranque comun	index.js	creacion de <code>app</code> , bases NeDB, registro de APIs, <code>express.static</code> , fallback SPA
Backend CRUD v2	src/back/v2/citys-stats.js	<code>removeDatabaseId</code> , <code>hasExactCityFields</code> , <code>normalizeCountryForStorage</code> , <code>parseCityStat</code> , <code>normalizeCityStat</code> , handlers <code>GET</code> , <code>POST</code> , <code>PUT</code> , <code>DELETE</code>
Backend integraciones v1	src/back/v1/citys-stats.js	<code>parseLimit</code> , <code>normalizeCountryKey</code> , <code>buildCityCountrySummaries</code> , <code>fetchJson</code> , <code>safeExternal</code> , <code>buildIntegratedCity</code>
Service base	frontend-group/src/services/apiBase.js	<code>API_ORIGIN</code> , <code>apiPath</code>
Service CRUD	frontend-group/src/services/citysStatsApi.js	<code>buildUrl</code> , <code>encodePathValue</code> , <code>friendlyApiMessage</code> , <code>handleResponse</code> , <code>getAllCityStats</code> , <code>createCityStat</code> , <code>updateCityStat</code>
Service integraciones	frontend-group/src/services/citysStatsIntegrations.js	<code>handleResponse</code> , <code>getCityStatsIntegrationSummary</code> , endpoints proxy individuales para pruebas
Lista paises LCC	frontend-group/src/lib/supportedCountries.js	<code>supportedCountries</code> , <code>normalizeSupportedCountry</code> , <code>isSupportedCountry</code>
CRUD Svelte	frontend-group/src/routes/citys-stats/+page.svelte	<code>validateCityStatForm</code> , <code>buildSearchQuery</code> , <code>refreshList</code> , <code>handleCreate</code> , <code>handleSearch</code> , <code>openEdit</code>
Edicion Svelte	frontend-group/src/routes/citys-stats/editar/[city]/[country]/+page.svelte	<code>loadResource</code> , <code>isSameResource</code> , <code>handleUpdate</code> , <code>updateRoute</code>
Analytics LCC	frontend-group/src/routes/analytics/citys-stats/+page.svelte	<code>loadHighcharts</code> , <code>renderChart</code> , <code>loadAnalytics</code>
Mapa LCC	frontend-group/src/routes/analytics/citys-stats/map/+page.svelte	<code>keyFor</code> , <code>colorFor</code> , <code>radiusFor</code> , <code>renderMap</code> , <code>loadMapData</code>
Integraciones UI	frontend-group/src/routes/integrations/citys-stats/+page.svelte	<code>sourceLabel</code> , <code>collectSummaryErrors</code> , <code>summaryDataset</code> , <code>loadHighcharts</code> , <code>createChart</code> , <code>renderIntegrationCharts</code> , <code>loadIntegrations</code>

16.3 Backend CRUD v2: contrato principal

Archivo principal:

```
src/back/v2/citys-stats.js
```

Esta es la API principal de LCC. Se usa para crear, listar, buscar, editar y borrar `citys-stats`.

removeDatabaseId

Codigo:

```
function removeDatabaseId(doc) {
  if (!doc) return doc;
  const { _id, ...rest } = doc;
  return rest;
}
```

Que hace:

Quita `_id`, que es un campo interno de NeDB, antes de devolver el JSON al cliente. La API publica solo debe exponer `city`, `country` y `un_2025_population`.

Como te pueden pedir cambiarla:

- Si quieren ocultar otro campo interno, se anade al destructuring: `const { _id, _rev, ...rest } = doc`.
- Si quieren devolver `_id` para depurar, se deja de usar esta funcion en las respuestas, aunque no es recomendable para el contrato publico.

Respuesta de defensa:

❗ NeDB necesita `_id` para guardar documentos, pero ese campo no forma parte del contrato REST de `citys-stats`.

hasExactCityFields

Codigo:

```
function hasExactCityFields(body) {
  if (!body || typeof body !== "object" || Array.isArray(body)) return false;

  const expected = ["city", "country", "un_2025_population"].sort();
  const keys = Object.keys(body).sort();

  return keys.length === expected.length &&
    keys.every((k, i) => k === expected[i]);
}
```

Que hace:

Comprueba que el body tenga exactamente los campos permitidos. No acepta campos de menos ni campos de mas.

Como te pueden pedir cambiarla:

- Anadir `continent`: meterlo en `expected`.
- Hacer un campo opcional: separar campos obligatorios y opcionales en vez de exigir igualdad exacta.
- Aceptar campos extra: quitar la comparacion estricta, aunque se pierde control del contrato.

Pillada probable:

❗ Si mando `{ city, country, un_2025_population, extra }`, devuelve `400` porque la API valida estructura exacta.

normalizeCountryForStorage

Codigo clave:

```
function normalizeCountryForStorage(country) {
  const normalized = String(country ?? "")
    .trim()
    .normalize("NFD")
    .replace(/[\u0300-\u036f]/g, "")
    .toLowerCase()
    .replace(/['.]/g, "")
    .replace(/&/g, "and")
    .replace(/^[a-z]+/g, "-")
    .replace(/^-+|-+$/g, "");

  return COUNTRY_ALIASES[normalized] || normalized;
}
```

Que hace:

Convierte el pais escrito por el usuario a la forma canonica que guarda la API. Por ejemplo, `Espana`, `espana` y `spain` acaban como `spain`; `South Korea` acaba como `south-korea`; `USA` acaba como `united-states`.

Por que existe:

Evita que el mismo pais se guarde con variantes distintas y permite comparar contra `SUPPORTED_COUNTRIES`.

Respuesta de defensa:

! Esta funcion no decide si el pais existe. Solo limpia y normaliza el texto. La decision de aceptar o rechazar se hace despues, comparando con `SUPPORTED_COUNTRIES`.

parseCityStat

Codigo clave:

```
function parseCityStat(body) {
  if (!hasExactCityFields(body)) {
    return { error: "JSON body does not match expected structure" };
  }

  const city = String(body.city).trim().toLowerCase();
  const country = normalizeCountryForStorage(body.country);
  const un_2025_population = Number(body.un_2025_population);

  if (
    !city ||
    !country ||
    !Number.isInteger(un_2025_population) ||
    un_2025_population <= 0
  ) {
    return { error: "JSON body does not match expected structure" };
  }

  if (!SUPPORTED_COUNTRIES.has(country)) {
    return { error: "Invalid country" };
  }

  return { item: { city, country, un_2025_population } };
}
```

Que hace:

Es la validacion real del recurso. Comprueba que el JSON tenga solo `city`, `country` y `un_2025_population`; convierte valores; exige poblacion entera positiva; y rechaza paises no soportados.

Como te pueden pedir cambiarla:

- Permitir poblacion `0`: cambiar `un_2025_population <= 0` por `un_2025_population < 0`.
- Aceptar un pais nuevo real: anadirlo a `SUPPORTED_COUNTRIES`.
- Aceptar una variante escrita por usuarios: anadirla a `COUNTRY_ALIASES`.
- Anadir un campo: actualizar `hasExactCityFields`, leerlo aqui, validarlo y devolverlo en `item`.

Respuesta de defensa:

! El frontend tambien valida, pero `parseCityStat` es la barrera real. Aunque llamen a la API con Postman, el backend no guarda datos invalidos ni paises que luego rompan integraciones externas.

`normalizeCityStat`

Codigo actual:

```
function normalizeCityStat(body) {
  return parseCityStat(body).item || null;
}
```

Que hace:

Es un envoltorio compatible con el codigo anterior. Antes normalizaba todo directamente; ahora delega en `parseCityStat`, que puede devolver errores mas concretos.

Respuesta de defensa:

! `normalizeCityStat` se mantiene como funcion auxiliar simple, pero el flujo nuevo usa `parseCityStat` para distinguir entre estructura invalida e `Invalid country`.

Handler `GET /api/v2/citys-stats`

Fragmento clave 1, lectura, filtros y ordenacion:

```

app.get(BASE_API_URL, (req, res) => {
  db.find({}, (err, docs) => {
    if (err) return res.sendStatus(500);

    let result = docs.map(removeDatabaseId);

    if (req.query.city !== undefined) {
      result = result.filter(
        d => d.city === String(req.query.city).trim().toLowerCase()
      );
    }

    if (req.query.country !== undefined) {
      result = result.filter(
        d => d.country === String(req.query.country).trim().toLowerCase()
      );
    }

    if (req.query.un_2025_population !== undefined) {
      const value = Number(req.query.un_2025_population);
      if (!Number.isFinite(value)) {
        return res.status(400).json({ error: "Invalid query" });
      }
      result = result.filter(d => d.un_2025_population === value);
    }

    if (req.query.q !== undefined) {
      const q = String(req.query.q).trim().toLowerCase();
      result = result.filter(d =>
        d.city.includes(q) || d.country.includes(q)
      );
    }
  });
});

```

Fragmento clave 2, ordenacion y paginacion dentro del mismo handler:

```

if (req.query.sort !== undefined) {
  const sort = String(req.query.sort).trim();
  let field = sort;
  let direction = 1;

  if (sort.startsWith("-")) {
    field = sort.slice(1);
    direction = -1;
  }

  const allowedFields = ["city", "country", "un_2025_population"];
  if (!allowedFields.includes(field)) {
    return res.status(400).json({ error: "Invalid sort field" });
  }

  result.sort((a, b) => {
    if (a[field] < b[field]) return -1 * direction;
    if (a[field] > b[field]) return 1 * direction;
    return 0;
  }));
}

let offset = 0;
let limit = result.length;

if (req.query.offset !== undefined) {
  offset = Number(req.query.offset);
  if (!Number.isInteger(offset) || offset < 0) {
    return res.status(400).json({ error: "Invalid offset" });
  }
}

if (req.query.limit !== undefined) {
  limit = Number(req.query.limit);
  if (!Number.isInteger(limit) || limit < 0) {
    return res.status(400).json({ error: "Invalid limit" });
  }
}

return res.status(200).json(result.slice(offset, offset + limit));

```

Que hace:

Lee todos los documentos, quita `_id`, aplica filtros exactos, busqueda libre, ordenacion y paginacion.

Orden real:

1. `db.find({})` lee NeDB.
2. `removeDatabaseId` limpia respuesta.
3. Filtra por `city`, `country` y `un_2025_population`.
4. Aplica `q` sobre ciudad o pais.
5. Ordena con lista blanca de campos.
6. Aplica `offset` y `limit`.
7. Devuelve `200` con array JSON.

Como te pueden pedir cambiarla:

- Filtro `min_population`: anadir bloque con `Number(req.query.min_population)` y `>=`.
- Ordenar por campo nuevo: anadirlo a `allowedFields`.
- Buscar tambien por otro campo: meterlo dentro del filtro de `q`.
- Cambiar paginacion por defecto: tocar `offset` y `limit`.

Pillada probable:

! Los query params llegan como texto, por eso se convierten y validan antes de filtrar.

Handler POST /api/v2/citys-stats

Codigo:

```
app.post(BASE_API_URL, (req, res) => {
  const parsed = parseCityStat(req.body);
  const item = parsed.item;

  if (parsed.error) {
    return res.status(400).json({ error: parsed.error });
  }

  db.findOne({ city: item.city, country: item.country }, (err, doc) => {
    if (err) return res.sendStatus(500);
    if (doc) return res.status(409).json({ error: "Resource already exists" });

    db.insert(item, (err2, newDoc) => {
      if (err2) return res.sendStatus(500);
      return res.status(201).json(removeDatabaseId(newDoc));
    });
  });
});
```

Que hace:

Crea un registro nuevo si el body es valido y no existe ya la pareja `city + country`.

Como te pueden pedir cambiarla:

- Cambiar la clave de duplicado: modificar el objeto de `db.findOne`.
- Devolver otro codigo al crear: cambiar `res.status(201)`.
- Guardar un campo nuevo: primero debe salir de `parseCityStat`.

Respuesta de defensa:

! `POST` crea dentro de la coleccion. Si el body es invalido o el pais no esta soportado, devuelve `400`. Si ya existe el recurso, devuelve `409 Conflict`.

Handler PUT /api/v2/citys-stats/:city/:country

Fragmento clave:

```

app.put(`${BASE_API_URL}/${city}/${country}`, (req, res) => {
  const city = req.params.city.trim().toLowerCase();
  const country = normalizeCountryForStorage(req.params.country);

  const parsed = parseCityStat(req.body);
  const item = parsed.item;

  if (parsed.error) {
    return res.status(400).json({ error: parsed.error });
  }

  if (item.city !== city || item.country !== country) {
    return res.status(400).json({ error: "URL and body do not match" });
  }

  db.findOne({ city, country }, (err, doc) => {
    if (err) return res.sendStatus(500);
    if (!doc) return res.status(404).json({ error: "Resource not found" });

    db.update({ city, country }, item, {}, (err2) => {
      if (err2) return res.sendStatus(500);
      db.findOne({ city, country }, (err3, updated) => {
        if (err3) return res.sendStatus(500);
        return res.status(200).json(removeDatabaseId(updated));
      });
    });
  });
});

```

Que hace:

Actualiza un recurso identificado por `city + country`. Exige que la URL y el body coincidan para evitar modificar un recurso distinto al solicitado.

Como te pueden pedir cambiarla:

- Permitir cambiar ciudad o país con `PUT`: quitar la comprobación URL/body y revisar duplicados. En este proyecto se resuelve desde frontend creando nuevo y borrando antiguo.
- Cambiar clave del recurso: tocar parámetros de ruta, `db.findOne`, `db.update`, `deleteCityStat`, `getOneCityStat` y rutas de edición.
- Devolver `204`: cambiar respuesta final, pero pierdes el objeto actualizado.

Respuesta de defensa:

! `PUT` actualiza un recurso concreto. La URL identifica el recurso y el body debe describir ese mismo recurso.

Handlers `DELETE`

Código:

```
app.delete(BASE_API_URL, (req, res) => {
  db.remove({}, { multi: true }, (err) => {
    if (err) return res.sendStatus(500);
    return res.sendStatus(204);
  });
});

app.delete(`${BASE_API_URL}/:city/:country`, (req, res) => {
  const city = req.params.city.trim().toLowerCase();
  const country = req.params.country.trim().toLowerCase();

  db.remove({ city, country }, {}, (err, numRemoved) => {
    if (err) return res.sendStatus(500);
    if (numRemoved === 0) {
      return res.status(404).json({ error: "Resource not found" });
    }

    return res.sendStatus(204);
  });
});
```

Que hacen:

El primer `DELETE` borra toda la coleccion. El segundo borra un registro concreto.

Como te pueden pedir cambiarlos:

- Devolver el elemento borrado: habria que buscarlo antes de borrar.
- Prohibir borrado total: cambiar el handler de coleccion a `405` o eliminar el boton del frontend.
- Borrado logico: anadir campo `deleted: true` en vez de `db.remove`.

16.4 Backend integraciones v1

Archivo principal:

```
src/back/v1/citys-stats.js
```

Aunque el CRUD principal esta en v2, las integraciones de LCC estan en v1 porque ahi se concentraron los endpoints proxy hacia APIs externas.

`parseLimit`

Codigo:

```
function parseLimit(value, fallback, max) {
  if (value === undefined) return fallback;

  const limit = Number(value);

  if (!Number.isInteger(limit) || limit < 1 || limit > max) {
    return null;
  }

  return limit;
}
```

Que hace:

Valida cuantos resultados puede pedir una integracion. Evita valores negativos, decimales, texto o limites demasiado grandes.

Como te pueden pedir cambiarla:

- Permitir mas elementos: subir `max` en la llamada, no necesariamente en la funcion.
- Permitir `0`: cambiar `limit < 1` por `limit < 0`, aunque para integraciones normalmente no tiene sentido.
- Cambiar valor por defecto: tocar el `fallback` en cada endpoint.

normalizeCountryKey

Codigo:

```
function normalizeCountryKey(value) {  
  return String(value ?? "")  
    .trim()  
    .normalize("NFD")  
    .replace(/[\u0300-\u036f]/g, "")  
    .replace(/[-_]+/g, " ")  
    .toLowerCase();  
}
```

Que hace:

Convierte paises a una clave comparable aunque lleguen con mayusculas, tildes, guiones o guiones bajos.

Como te pueden pedir cambiarla:

- Si una API usa nombres raros, anadir una tabla de equivalencias antes de devolver la clave.
- Si se quiere integrar por ciudad, crear una funcion equivalente para ciudades y revisar todos los cruces por pais.

Respuesta de defensa:

! Las APIs externas no escriben paises exactamente igual. Normalizar reduce fallos de coincidencia.

buildCityCountrySummaries

Fragmento clave:

```
function buildCityCountrySummaries(items) {
  const byCountry = new Map();

  items.forEach((item) => {
    const key = normalizeCountryKey(item.country);
    if (!key) return;

    const population = readFiniteNumber(item.un_2025_population, 0);
    const current = byCountry.get(key) || {
      country: item.country,
      countryKey: key,
      city: item.city,
      topCity: item.city,
      topCityPopulation: population,
      cityCount: 0,
      un_2025_population: 0,
      cities: []
    };

    current.cityCount += 1;
    current.un_2025_population += population;
    current.cities.push({ city: item.city, population });

    if (population > current.topCityPopulation) {
      current.city = item.city;
      current.topCity = item.city;
      current.topCityPopulation = population;
    }

    byCountry.set(key, current);
  });

  return [...byCountry.values()]
    .map((item) => ({
      ...item,
      cities: item.cities.sort((a, b) => b.population - a.population)
    }))
    .sort((a, b) => b.un_2025_population - a.un_2025_population);
}
```

Que hace:

Agrupar ciudades locales por país. Calcular cuántas ciudades hay, población agregada y ciudad principal. Es la base para cruzar con REST Countries, World Bank y APIs SOS.

Como te pueden pedir cambiarla:

- Calcular media: añadir `averagePopulation = un_2025_population / cityCount`.
- Integrar por ciudad: esta función dejaría de ser la base y habría que cambiar endpoints y widgets.
- Mostrar todos los países sin ordenar: quitar o cambiar el `sort` final.

fetchJson

Código:

```

async function fetchJson(url, sourceName, timeoutMs = 20000) {
  const controller = new AbortController();
  const timeout = setTimeout(() => controller.abort(), timeoutMs);

  try {
    const response = await fetch(url, {
      headers: {
        Accept: "application/json",
        "User-Agent": "SOS2526-29 citys-stats integration"
      },
      signal: controller.signal
    });

    const text = await response.text();
    let data = null;

    try {
      data = text ? JSON.parse(text) : null;
    } catch {
      throw new Error(`${sourceName} did not return JSON`);
    }

    if (!response.ok) {
      const reason = data?.message || data?.error || response.statusText;
      throw new Error(`${sourceName} returned ${response.status}: ${reason}`);
    }

    return data;
  } finally {
    clearTimeout(timeout);
  }
}

```

Que hace:

Llama a APIs externas con timeout, cabeceras, lectura de texto, parseo JSON y control de errores HTTP.

Como te pueden pedir cambiarla:

- Aumentar timeout: cambiar el tercer parametro o el valor por defecto.
- Consumir API con token: anadir cabecera `Authorization`.
- Aceptar CSV o texto: cambiar la parte de `JSON.parse`.

Respuesta de defensa:

❗ Centralizar `fetchJson` evita repetir control de timeout y errores en cada integracion.

`safeExternal`

Codigo:

```

async function safeExternal(source, task) {
  try {
    return { source, data: await task(), error: null };
  } catch (err) {
    return { source, data: null, error: err.message };
  }
}

```

Que hace:

Convierte una llamada externa en un objeto controlado. Si falla, no rompe todo el resumen; devuelve `error` para mostrarlo como fallo parcial.

Como te pueden pedir cambiarla:

- Hacer que falle toda la integracion si falla una API: dejar que el error se lance sin envolverlo.
- Guardar mas detalle: anadir `status`, `url` o `timestamp` al objeto devuelto.

buildIntegratedCity

Fragmento clave:

```
function buildIntegratedCity(base, worldBankByCode, worldBankBatchError, studentApis) {
  const code = base.countryResult.data?.cca3;
  let worldBankResult;

  if (!code) {
    worldBankResult = {
      source: "World Bank Indicators API",
      data: null,
      error: "Country ISO3 code not available"
    };
  } else if (worldBankBatchError) {
    worldBankResult = {
      source: "World Bank Indicators API",
      data: null,
      error: worldBankBatchError
    };
  } else {
    const data = worldBankByCode.get(code) ?? null;
    worldBankResult = {
      source: "World Bank Indicators API",
      data,
      error: data ? null : "World Bank data not found"
    };
  }

  return {
    city: base.item.city,
    country: base.item.country,
    cityCount: base.item.cityCount ?? 1,
    topCity: base.item.topCity ?? base.item.city,
    topCityPopulation: base.item.topCityPopulation ?? base.item.un_2025_population,
    un_2025_population: base.item.un_2025_population,
    geocoding: base.geocodingResult.data,
    countryInfo: base.countryResult.data,
    worldBankPopulation: worldBankResult.data,
    integrationErrors: [base.geocodingResult, base.countryResult, worldBankResult]
      .filter((result) => result.error)
      .map((result) => ({ source: result.source, error: result.error }));
  };
}
```

Que hace:

Monta el objeto final del endpoint `/api/v1/citys-stats/integrations/summary`: datos locales, geocoding, pais, World Bank y errores parciales.

Como te pueden pedir cambiarla:

- Anadir una API externa nueva al resumen: calcularla antes y meter su dato aqui.
- Cambiar el formato JSON del resumen: cambiar las propiedades devueltas y actualizar service/pantalla.
- Mostrar mas errores parciales: anadir mas resultados a la lista `integrationErrors`.

16.5 Services frontend

Los services separan Svelte de los detalles HTTP. La pantalla no debe construir URLs complejas ni traducir todos los codigos de error.

apiPath

Archivo:

frontend-group/src/services/apiBase.js

Codigo:

```
export const API_ORIGIN = import.meta.env.DEV
  ? "http://localhost:10000"
  : window.location.origin;

export function apiPath(path) {
  return `${API_ORIGIN}${path}`;
}
```

Que hace:

En desarrollo llama al backend Express del puerto `10000`. En produccion usa el mismo origen que la web desplegada.

Como te pueden pedir cambiarla:

- Cambiar puerto local: modificar `http://localhost:10000`.
- Usar variable de entorno: leer `import.meta.env.VITE_API_ORIGIN`.
- Desplegar frontend y backend separados: cambiar `API_ORIGIN` de produccion.

buildUrl

Archivo:

frontend-group/src/services/citysStatsApi.js

Codigo:

```
function buildUrl(path = "", query = {}) {
  const url = new URL(`${CITYS_STATS_API_BASE}${path}`, window.location.origin);

  Object.entries(query).forEach(([key, value]) => {
    if (value === undefined || value === null || value === "") {
      return;
    }

    url.searchParams.set(key, value);
  });

  return url.toString();
}
```

Que hace:

Crea la URL del CRUD y anade query params sin concatenar strings a mano.

Como te pueden pedir cambiarla:

- Pasar de v2 a v3: cambiar `CITYS_STATS_API_BASE`, no esta funcion.
- Parametros repetidos: cambiar `set` por `append`.
- Enviar valores vacios: quitar el filtro de `undefined`, `null` y `""`.

handleResponse

Codigo:

```

async function handleResponse(response) {
  if (response.status === 204) {
    return null;
  }

  const text = await response.text();
  let data = null;

  try {
    data = text ? JSON.parse(text) : null;
  } catch {
    data = text;
  }

  if (!response.ok) {
    throw new Error(friendlyApiMessage(response.status, data?.error));
  }

  return data;
}

```

Que hace:

Convierte respuestas `fetch` en datos o en `Error`. Las pantallas solo necesitan `try/catch`.

Como te pueden pedir cambiarla:

- Cambiar mensajes visibles: tocar `friendlyApiMessage`.
- Mostrar el error exacto del backend: lanzar `data?.error` directamente.
- Soportar otro tipo de respuesta: cambiar la lectura de `text` y parseo.

Funciones CRUD exportadas

Codigo:

```

export async function getAllCityStats(query = {}) {
  const response = await fetch(buildUrl("", query));
  return handleResponse(response);
}

export async function createCityStat(cityStat) {
  const response = await fetch(buildUrl(), {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(cityStat)
  });
  return handleResponse(response);
}

export async function updateCityStat(city, country, cityStat) {
  const response = await fetch(
    buildUrl(`/${encodeURIComponent(city)}/${encodeURIComponent(country)}`),
    {
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(cityStat)
    }
  );
  return handleResponse(response);
}

```

Que hacen:

Son la puerta de entrada del frontend al CRUD. Cada una representa una accion REST: listar, crear o actualizar.

Como te pueden pedir cambiarlas:

- Endpoint nuevo: crear una funcion nueva aqui y llamarla desde la pantalla.
- Header nuevo: anadirlo en el `fetch` correspondiente.
- Cambiar version API: revisar `CITYS_STATS_API_BASE`.

16.6 Pantallas Svelte principales

`validateCityStatForm`

Archivo:

`frontend-group/src/routes/citys-stats/+page.svelte`

Codigo:

```
function validateCityStatForm(form) {
  const city = String(form.city ?? "").trim();
  const country = String(form.country ?? "").trim();
  const un_2025_population = parsePositiveInteger(
    form.un_2025_population,
    "Poblacion estimada en 2025"
  );

  if (!city) {
    throw new Error("Indique una ciudad.");
  }

  if (!country) {
    throw new Error("Indique un pais.");
  }

  return { city, country, un_2025_population };
}
```

Que hace:

Valida el formulario antes del `POST`. Mejora experiencia de usuario, pero no sustituye a la validacion del backend.

Como te pueden pedir cambiarla:

- Anadir campo `continent`: leerlo, validar que no este vacio y devolverlo.
- Permitir decimales o cero: tocar `parsePositiveInteger`.
- Cambiar mensaje visible: modificar los `throw new Error`.

`buildSearchQuery`

Codigo:

```
function buildSearchQuery() {
  const query = {
    q: String(searchForm.q ?? "").trim(),
    city: String(searchForm.city ?? "").trim(),
    country: String(searchForm.country ?? "").trim(),
    un_2025_population: parseOptionalPositiveInteger(
      searchForm.un_2025_population,
      "Poblacion exacta"
    ),
    sort: String(searchForm.sort ?? "").trim(),
    limit: parseOptionalNonNegativeInteger(searchForm.limit, "Numero maximo de resultados"),
    offset: parseOptionalNonNegativeInteger(searchForm.offset, "Posicion inicial")
  };

  Object.keys(query).forEach((key) => {
    if (query[key] === "" || query[key] === undefined) {
      delete query[key];
    }
  });

  return query;
}
```

Que hace:

Convierte inputs de busqueda en query params limpios para `GET /api/v2/citys-stats`.

Como te pueden pedir cambiarla:

- Filtro `min_population`: anadir campo en `emptySearchForm`, input HTML y propiedad aqui.
- Nuevo orden: anadir opcion en `sortOptions`; si es campo nuevo, tambien backend.
- Mandar campos vacios: quitar el `delete`, aunque ahora se evita ruido.

refreshList

Codigo:

```
async function refreshList(query = activeQuery, successMessage = "") {
  loading = true;
  error = "";

  try {
    citysStats = await getAllCitysStats(query);
    activeQuery = { ...query };

    if (successMessage) {
      message = successMessage;
    }

    return citysStats;
  } catch (e) {
    citysStats = [];
    error = e.message || "No se pudieron cargar los datos de ciudades.";
    return [];
  } finally {
    loading = false;
  }
}
```

Que hace:

Recarga tabla, mantiene filtros activos y controla estados `loading`, `message` y `error`.

Como te pueden pedir cambiarla:

- Mantener tabla si falla: no vaciar `citysStats` en `catch`.
- Mostrar contador total: usar `citysStats.length` o crear endpoint `/count`.
- Recargar sin filtros despues de crear: llamar con `{}` en vez de `activeQuery`.

handleCreate

Codigo:

```

async function handleCreate() {
  clearFeedback();

  try {
    const payload = validateCityStatForm(createForm);
    const created = await createCityStat(payload);

    createForm = emptyCreateForm();
    await refreshList(
      activeQuery,
      `Se ha creado el registro de ${created.city} (${created.country}).`
    );
  } catch (e) {
    error = e.message || "No se pudo crear el registro.";
  }
}

```

Que hace:

Representa el flujo del boton de crear: limpiar mensajes, validar, llamar service, limpiar formulario y recargar tabla.

Como te pueden pedir cambiarla:

- Redirigir a edicion tras crear: llamar a `openEdit(created.city, created.country)`.
- No limpiar formulario: quitar `createForm = emptyCreateForm()`.
- Mostrar mensaje distinto: cambiar el texto enviado a `refreshList`.

handleUpdate**Archivo:**

```
frontend-group/src/routes/citys-stats/editar/[city]/[country]/+page.svelte
```

Fragmento clave:

```

async function handleUpdate() {
  clearFeedback();

  try {
    const payload = validateForm();

    if (isSameResource(payload)) {
      const updated = await updateCityStat(
        originalKey.city,
        originalKey.country,
        payload
      );

      form = {
        city: updated.city,
        country: updated.country,
        un_2025_population: updated.un_2025_population
      };

      message = "Los cambios se han guardado correctamente.";
      return;
    }

    const created = await createCityStat(payload);
    await deleteCityStat(originalKey.city, originalKey.country);

    originalKey = { city: created.city, country: created.country };
    updateRoute(created.city, created.country);
  } catch (e) {
    error = e.message || "No se pudieron guardar los cambios.";
  }
}

```

Que hace:

Si la clave `city + country` no cambia, usa `PUT`. Si cambia, crea el nuevo recurso y borra el anterior.

Como te pueden pedir cambiarla:

- Hacer siempre `PUT`: solo si el backend permite cambiar clave en `PUT`.
- Pedir confirmacion antes de borrar: anadir confirmacion antes de `deleteCityStat`.
- Cambiar la ruta tras editar: tocar `updateRoute`.

Respuesta de defensa:

! Como `city + country` identifica el recurso, cambiar la clave equivale a moverlo. Por eso el frontend crea el nuevo y borra el antiguo.

16.7 Analytics, mapa e integraciones UI

`renderChart` de analytics individual

Codigo resumido:

```
function renderChart() {
  if (!chartContainer) return;

  const data = citysStats
    .slice()
    .sort((a, b) => Number(b.un_2025_population) - Number(a.un_2025_population))
    .map((item) => ({
      name: labelFor(item),
      y: Number(item.un_2025_population)
    }));

  chart?.destroy();

  chart = Highcharts.chart(chartContainer, {
    chart: { type: "pie", backgroundColor: "transparent" },
    series: [{ name: "Poblacion 2025", colorByPoint: true, data }]
  });
}
```

Que hace:

Prepara datos de poblacion y pinta una grafica Highcharts. Destruye la anterior para no duplicar instancias.

Como te pueden pedir cambiarla:

- Cambiar `pie` a `bar`: cambiar `chart.type`, `xAxes.categories` y forma de `series`.
- Mostrar top 5: aplicar `.slice(0, 5)` despues de ordenar.
- Cambiar metrica: mapear otro campo en `y`.

renderMap

Fragmento clave:

```
function renderMap() {
  if (!mapContainer || !Highcharts || points.length === 0) return;

  mapChart?.destroy();

  mapChart = Highcharts.mapChart(mapContainer, {
    chart: { map: worldMap, backgroundColor: "transparent" },
    series: [
      { name: "Países", nullColor: "#dbe6dd" },
      {
        type: "mappoint",
        name: "Ciudades",
        data: points.map((point) => ({
          name: titleCase(point.city),
          lat: point.lat,
          lon: point.lon,
          marker: { radius: point.radius, fillColor: point.color },
          custom: point
        })))
      ]
    ]
  });
}
```

Que hace:

Crea un mapa mundial con marcadores para ciudades geolocalizadas.

Como te pueden pedir cambiarla:

- Anadir ciudad al mapa: tocar `coordinates`, no solo `renderMap`.
- Cambiar color: tocar `colorFor` o `colorAxis`.
- Cambiar tamano: tocar `radiusFor`.
- Cambiar tooltip: tocar `tooltip.pointFormatter`.

collectSummaryErrors y summaryDataset

Archivo:

frontend-group/src/routes/integrations/citys-stats/+page.svelte

Codigo:

```
function collectSummaryErrors(summary, items) {
  const errors = [];
  const seen = new Set();

  items.forEach((item) => {
    (item.integrationErrors ?? []).forEach((error) => {
      // Normaliza y deduplica errores parciales.
    });
  });

  (summary?.studentApis ?? []).forEach((api) => {
    // Incluye errores generales de APIs SOS externas.
  });

  return errors;
}

function summaryDataset(summary, key, metric, limit = 8) {
  const rows = Array.isArray(summary?.studentApiDatasets?.[key])
    ? summary.studentApiDatasets[key]
    : [];

  return rows
    .filter((row) => numberOrNull(row?.[metric]) !== null)
    .slice(0, limit);
}
```

Que hace:

- `collectSummaryErrors` recoge errores parciales que ya vienen del backend y evita mostrar duplicados.
- `summaryDataset` extrae rankings externos del objeto `studentApiDatasets` que devuelve el endpoint `summary`.
- Estas funciones sustituyen al flujo antiguo donde el navegador llamaba una a una a cada API externa.

Como te pueden pedir cambiarla:

- Mostrar mas informacion de errores: ampliar los objetos que mete `collectSummaryErrors`.
- Cambiar el numero de paises mostrados: modificar el `limit` usado al llamar a `summaryDataset`.
- Cambiar una metrica externa: cambiar el nombre de `metric` que se pasa a `summaryDataset`.

loadIntegrations

Fragmento clave:

```

async function loadIntegrations() {
  loading = true;
  error = "";
  integrationErrors = [];
  restoredInitialData = false;
  destroyIntegrationCharts();

  try {
    let summary = await getCityStatsIntegrationSummary(selectedLimit);
    countrySummaries = Array.isArray(summary?.items) ? summary.items : [];

    if (countrySummaries.length === 0) {
      await loadInitialCityStats();
      summary = await getCityStatsIntegrationSummary(selectedLimit);
      countrySummaries = Array.isArray(summary?.items) ? summary.items : [];
      restoredInitialData = countrySummaries.length > 0;
    }

    geocodingRows = countrySummaries.map((item) => ({ ...item }));
    countryCards = countrySummaries.map((item) => ({
      ...item,
      countryData: item.countryInfo
    }));
    worldBankRows = countrySummaries.map((item) => ({
      country: item.country,
      localPopulation: item.un_2025_population,
      countryInfo: item.countryInfo,
      worldBank: item.worldBankPopulation
    }));

    touristCountries = summaryDataset(summary, "touristCountries", "totalArrivals", 8);
    integrationErrors = collectSummaryErrors(summary, countrySummaries);

    loading = false;
    await tick();
    await loadHighcharts();
    renderIntegrationCharts();
  } catch (e) {
    error = e.message || "No se pudieron cargar las integraciones.";
    loading = false;
  }
}

```

Que hace:

Orquesta toda la vista de integraciones: pide un unico resumen al backend, adapta ese resumen a las variables que usan los widgets, registra errores parciales, espera al DOM y pinta Highcharts.

Como te pueden pedir cambiarla:

- Anadir API externa: anadirla al backend `summary`, devolver su dataset y leerlo en la pantalla con `summaryDataset`.
- Cambiar limite: tocar `selectedLimit` y el selector HTML.
- Quitar autocarga inicial: eliminar el bloque `if (countrySummaries.length === 0)`.

Respuesta de defensa:

- ! Antes la pantalla lanzaba muchas peticiones. Ahora el frontend hace una unica llamada a `summary`; el backend coordina las integraciones, controla errores parciales y devuelve datos listos para pintar.

16.8 Flujos completos que conviene memorizar

La chuleta completa esta en la seccion 17, para evitar duplicar informacion.

Idea que hay que memorizar:

- ! El orden real depende del contexto: pantalla, boton o endpoint. Las funciones no se ejecutan por estar arriba o abajo en el archivo, sino porque algo las llama.

16.9 Recetas de cambio que mas pueden pedir

Añadir campo `continent`

Backend v2:

```
const expected = ["city", "country", "continent", "un_2025_population"].sort();

const continent = String(body.continent).trim().toLowerCase();

if (!city || !country || !continent || !Number.isInteger(un_2025_population)) {
  return null;
}

return { city, country, continent, un_2025_population };
```

Tambien revisar:

- `initialData` en v1 y v2.
- `tests/LCC` porque `POST` y `PUT` necesitan el campo nuevo.
- `emptycreateForm`, `emptysearchform`, `validateCityStatForm` y HTML del CRUD.
- Pantalla de edicion: `emptyForm`, `loadResource`, `validateForm`, `handleUpdate`.
- Tablas, analytics, mapa e integraciones si se quiere mostrar o usar `continent`.

Frase de defensa:

- ! Como cambia el contrato del recurso, no basta con anadir un input. Hay que tocar backend, frontend y tests.

Añadir filtro `min_population`

Backend:

```
if (req.query.min_population !== undefined) {
  const value = Number(req.query.min_population);
  if (!Number.isFinite(value) || value < 0) {
    return res.status(400).json({ error: "Invalid query" });
  }
  result = result.filter(d => d.un_2025_population >= value);
}
```

Frontend:

```
min_population: parseOptionalNonNegativeInteger(
  searchForm.min_population,
  "Poblacion minima"
)
```

Tambien revisar:

- `emptySearchForm`.
- Input HTML de búsqueda.
- Tests de query valida e invalida.

Anadir endpoint `/api/v2/citys-stats/count`

Poner antes de la ruta parametrizada `/:city/:country`:

```
app.get(`${BASE_API_URL}/count`, (req, res) => {
  db.count({}, (err, count) => {
    if (err) return res.sendStatus(500);
    return res.status(200).json({ count });
  });
});
```

Por que antes:

Express evalua rutas en orden. Las rutas especiales se colocan antes de las rutas con parametros para evitar capturas accidentales.

Frontend:

```
export async function getCitysStatsCount() {
  const response = await fetch(buildUrl("/count"));
  return handleResponse(response);
}
```

Cambiar grafica `pie` a `bar`

No basta siempre con cambiar `type`. Para barras suele hacer falta eje X y serie numerica:

```
chart = Highcharts.chart(chartContainer, {
  chart: { type: "bar", backgroundColor: "transparent" },
  xAxis: {
    categories: data.map((item) => item.name)
  },
  series: [
    {
      name: "Poblacion 2025",
      data: data.map((item) => item.y)
    }
  ]
});
```

Revisar tooltip, altura del contenedor y tests visuales si los hay.

Anadir una API externa nueva

Backend:

1. Crear constante de URL externa.
2. Crear funcion que use `fetchJson`.
3. Normalizar la respuesta si viene con nombres raros.
4. Crear endpoint proxy en `src/back/v1/citys-stats.js`.
5. Si entra en resumen, meterla en `/integrations/summary`.

Service:

```
export async function getNuevaApi() {
  const response = await fetch(`${CITYS_STATS_INTEGRATIONS_API_BASE}/integrations/nueva-api`);
  return handleResponse(response);
}
```

Pantalla:

1. Hacer que el backend incluya la nueva API dentro de `/integrations/summary`.
2. Si el widget necesita ranking externo completo, devolverlo en `studentApiDatasets`.
3. En `loadIntegrations`, leerlo con `summaryDataset`.
4. Crear `renderNuevaApiChart`.
5. Anadirlo a `renderIntegrationCharts`.
6. Anadir panel HTML.
7. Destruir la grafica con las demas.

Frase de defensa:

- ! Primero backend proxy, luego resumen agregado, luego pantalla. Asi el navegador no depende directamente de la API externa y no dispara demasiadas peticiones.

16.10 Asincronia sin liarse

Reglas simples:

```
callback de NeDB: db.find(..., (err, docs) => { ... })
async/await: espera peticiones HTTP o imports dinamicos
Promise.all: lanza varias llamadas en paralelo y espera todas
tick: espera a que Svelte pinte el DOM antes de usar Highcharts
try/catch: convierte errores en mensajes controlados
finally: ejecuta limpieza aunque haya error
```

Frases utiles:

- `db.find` no devuelve los documentos directamente; los entrega en el callback.
- `await fetch(...)` espera la respuesta de red.
- `Promise.all` acelera integraciones independientes.
- `tick` evita pintar una grafica antes de que exista el contenedor.
- `chart?.destroy()` evita duplicar graficas al repintar.

16.11 Tabla de impacto: si tocas esto, revisa esto

Cambio	Revisar tambien
<code>hasExactCityFields</code>	<code>parseCityStat</code> , bodies de tests, formularios Svelte
<code>parseCityStat</code>	datos iniciales, mensajes de error, POST, PUT, edicion
<code>SUPPORTED_COUNTRIES</code> o alias	<code>supportedCountries.js</code> , frontend crear/editar, integraciones y tests
Handler <code>GET</code> v2	<code>buildSearchQuery</code> , tests de filtros, analytics y mapa
Handler <code>POST</code> v2	<code>createCityStat</code> , <code>handleCreate</code> , test de duplicado <code>409</code>
Handler <code>PUT</code> v2	<code>updateCityStat</code> , <code>handleUpdate</code> , regla URL/body
Handler <code>DELETE</code> v2	botones de borrado, mensajes, <code>204</code> y tests
<code>buildCityCountrySummaries</code>	integraciones por pais, <code>summary</code> , widgets y tablas
<code>fetchJson</code>	todos los endpoints externos
<code>safeExternal</code>	errores parciales del backend de integraciones
<code>apiPath</code>	todas las llamadas <code>fetch</code> del frontend
<code>handleResponse</code>	todos los mensajes de error visibles
<code>refreshList</code>	listado, busqueda, crear, borrar y cargar iniciales
<code>handleUpdate</code>	pantalla de edicion y regla de clave compuesta
<code>renderChart</code>	grafica individual, tooltip, <code>onDestroy</code>
<code>renderMap</code>	coordenadas, colores, radios, tooltip
<code>loadIntegrations</code>	todas las APIs externas y widgets Highcharts

16.12 Preguntas dificiles sobre funciones

Pregunta	Respuesta segura
Por que quitas <code>_id</code> ?	Porque es interno de NeDB y no pertenece al contrato publico.
Por que validas en frontend y backend?	Frontend mejora UX; backend protege el sistema real.
Por que no permites paises inventados?	Porque despues no tienen ISO3 real y rompen cruces con REST Countries o World Bank.
Que pasa si escribo <code>Espana</code> , <code>usa</code> o <code>uk</code> ?	<code>normalizeCountryForStorage</code> y los alias lo convierten al pais canonico antes de guardar.
Por que <code>POST</code> devuelve <code>409</code> ?	Porque ya existe un recurso con la misma clave <code>city + country</code> .
Por que <code>PUT</code> compara URL y body?	Para asegurar que actualizo exactamente el recurso identificado por la URL.
Por que integras por pais?	REST Countries, World Bank y varias APIs SOS encajan mejor por <code>country</code> que por ciudad.
Por que la pantalla usa <code>summary</code> ?	Para hacer una sola peticion desde el navegador y evitar picos de demasiadas peticiones.
Por que <code>safeExternal</code> no rompe todo?	Porque en integraciones interesa mostrar datos parciales aunque una fuente externa falle.
Por que <code>Promise.all</code> ?	En el backend permite consultar APIs independientes en paralelo y reducir espera.
Por que <code>tick</code> antes de Highcharts?	Porque Highcharts necesita que el contenedor ya exista en el DOM.
Donde cambiarias mensajes de error?	En <code>friendlyApiMessage</code> para CRUD y en <code>handleResponse</code> de integraciones si son externas.
Como buscas quien llama a una funcion?	Con <code>rg "nombreFuncion"</code> desde la raiz del repositorio.

16.13 Mini guion si te piden modificar algo en directo

1. Identificar capa: API, service, pantalla, grafica o test.
2. Buscar funcion con `rg`.
3. Hacer cambio minimo.
4. Revisar funciones que la llaman.
5. Ejecutar test o comprobacion manual.
6. Explicar que se ha cambiado y por que.

Comandos utiles:

```
rg "parseCityStat"
rg "SUPPORTED_COUNTRIES"
rg "buildSearchQuery"
rg "loadIntegrations"
npm.cmd run test-LCC-v2
npm.cmd run test-LCC-e2e
```

Regla final:

- ! Si cambia el contrato de datos, toca backend, frontend y tests. Si cambia solo la visualizacion, normalmente toca Svelte y quiza el service si necesita datos nuevos.

17. Flujos de ejecucion

Idea clave para defensa:

- ! El orden real no depende de donde este escrita la funcion, sino de quien la llama: una ruta de Express, un `onMount`, un boton o un service del frontend.

Otra idea importante:

- ! Cuando pongo "No hay funcion auxiliar", significa que ese paso esta escrito directamente dentro de la ruta o del componente. No te falta memorizar ninguna funcion con nombre.

Arranque general

1. El usuario ejecuta `npm start`.
2. Node ejecuta `index.js`.
3. Express crea `app`.
4. Se activan CORS y lectura JSON.
5. Se abren las bases NeDB.
6. Se importan APIs v1 y v2.
7. Cada API ejecuta `module.exports(app, db)`.
8. Cada API crea constantes, declara funciones y registra rutas.
9. Se sirve `public`.
10. Se registra el fallback de la SPA.
11. El servidor queda escuchando.

Importante:

- En el arranque las funciones auxiliares se declaran, pero no se ejecutan.
- Las rutas `app.get`, `app.post`, `app.put` y `app.delete` solo se registran.
- Una ruta se ejecuta solo cuando llega una peticion con su metodo y URL.

Regla para cualquier pantalla Svelte

1. El usuario abre una URL, por ejemplo `/citys-stats`.
2. Express devuelve `public/index.html`.
3. Arranca Svelte.
4. `App.svelte` mira `window.location.pathname`.
5. El router carga el componente de esa ruta.
6. El `<script>` del componente se ejecuta.
7. Las funciones se declaran.
8. `onMount` ejecuta la carga inicial si existe.
9. Los botones ejecutan sus handlers solo cuando el usuario pulsa.

Si estas en `/citys-stats`

Abrir la pantalla:

```
onMount
-> refreshList({}, "")
-> getAllCitysStats({})
-> buildUrl("", {})
-> fetch GET /api/v2/citys-stats
  -> backend v2 app.get(BASE_API_URL)
    -> db.find
    -> docs.map(removeDatabaseId)
    -> filtros exactos si vienen
      No hay funcion auxiliar: se hace dentro de la ruta.
    -> busqueda q si viene
      No hay funcion auxiliar: se hace dentro de la ruta.
    -> sort si viene
      No hay funcion auxiliar: se hace dentro de la ruta.
    -> offset/limit si vienen
      No hay funcion auxiliar: se hace dentro de la ruta.
    -> result.slice(offset, offset + limit)
    -> res.status(200).json(...)
-> handleResponse(response)
  -> response.text()
  -> JSON.parse si hay cuerpo
-> citysStats = datos recibidos
-> activeQuery = {}
-> loading = false
```

Buscar:

```

handleSearch
-> clearFeedback
-> buildSearchQuery
  -> parseOptionalPositiveInteger para un_2025_population
  -> parseOptionalNonNegativeInteger para limit
  -> parseOptionalNonNegativeInteger para offset
-> refreshList(query)
  -> getAllCityStats(query)
    -> buildUrl("", query)
    -> fetch GET /api/v2/citys-stats?...query
      -> backend v2 app.get(BASE_API_URL)
        -> db.find
        -> docs.map(removeDatabaseId)
        -> filtro city si viene
          No hay funcion auxiliar.
        -> filtro country si viene
          No hay funcion auxiliar.
        -> filtro un_2025_population si viene
          No hay funcion auxiliar.
        -> busqueda q si viene
          No hay funcion auxiliar.
        -> sort si viene
          No hay funcion auxiliar.
        -> offset/limit si vienen
          No hay funcion auxiliar.
        -> result.slice(...)
        -> res.status(200).json(...)
      -> handleResponse(response)
-> hasQueryValues(query)
-> message o error

```

Limpiar filtros:

```

handleResetSearch
-> emptySearchForm
-> clearFeedback
-> refreshList({})
  -> getAllCityStats({})
    -> buildUrl("", {})
    -> fetch GET /api/v2/citys-stats
    -> backend v2 app.get(BASE_API_URL)
    -> handleResponse(response)

```

Crear:

```

handleCreate
-> clearFeedback
-> validateCityStatForm
  -> parsePositiveInteger
-> createCityStat
  -> buildUrl()
  -> fetch POST /api/v2/citys-stats
    -> backend v2 app.post(BASE_API_URL)
    -> parseCityStat
      -> hasExactCityFields
      -> normalizeCountryForStorage
      -> SUPPORTED_COUNTRIES
    -> db.findOne para duplicado
    -> db.insert
    -> removeDatabaseId
    -> res.status(201).json(...)
  -> handleResponse(response)
-> emptyCreateForm
-> refreshList(activeQuery)
  -> getAllCityStats(activeQuery)
  -> buildUrl("", activeQuery)
  -> fetch GET /api/v2/citys-stats
  -> backend v2 app.get(BASE_API_URL)
  -> handleResponse(response)

```

Cargar datos iniciales:

```

handleLoadInitialData
-> clearFeedback
-> loadInitialCityStats
  -> buildUrl("/loadInitialData")
  -> fetch GET /api/v2/citys-stats/loadInitialData
    -> backend v2 app.get(`${BASE_API_URL}/loadInitialData`)
    -> db.count
    -> si count > 0:
      -> db.find
      -> docs.map(removeDatabaseId)
      -> res.status(200).json(...)
    -> si count === 0:
      -> db.insert(initialData)
      -> docs.map(removeDatabaseId)
      -> res.status(201).json(...)
  -> handleResponse(response)
-> refreshList(activeQuery)

```

Borrar uno:

```

handleDeleteOne
-> clearFeedback
-> deleteCityStat
  -> encodePathValue(city)
  -> encodePathValue(country)
  -> buildUrl("/:city/:country")
  -> fetch DELETE /api/v2/citys-stats/:city/:country
    -> backend v2 app.delete(`${BASE_API_URL}/:city/:country`)
    -> normaliza req.params.city y req.params.country
    -> db.remove({ city, country }, {})
    -> res.sendStatus(204)
    -> o res.status(404).json(...) si no existia
  -> handleResponse(response)
-> refreshList(activeQuery)

```

Borrar todos:

```

handleDeleteAll
-> clearFeedback
-> deleteAllCityStats
-> buildUrl()
-> fetch DELETE /api/v2/citys-stats
-> backend v2 app.delete(BASE_API_URL)
-> db.remove({}, { multi: true })
-> res.sendStatus(204)
-> handleResponse(response)
-> refreshList(activeQuery)

```

Abrir edicion:

```

openEdit
-> encodeURIComponent(city)
-> encodeURIComponent(country)
-> navigate("/citys-stats/editar/:city/:country")

```

Si estas en `/citys-stats/editar/:city/:country`

Abrir pantalla:

```

onMount
-> loadResource
-> getOneCityStat(params.city, params.country)
-> encodePathValue(city)
-> encodePathValue(country)
-> buildUrl("/:city/:country")
-> fetch GET /api/v2/citys-stats/:city/:country
-> backend v2 app.get(`${BASE_API_URL}/${city}/${country}`)
-> normaliza req.params.city y req.params.country
-> db.findOne({ city, country })
-> removeDatabaseId
-> res.status(200).json(...)
-> o 404 si no existe
-> handleResponse(response)
-> rellena form
-> rellena originalKey

```

Guardar sin cambiar `city` ni `country`:

```

handleUpdate
-> clearFeedback
-> validateForm
  -> parsePositiveInteger
-> isSameResource true
-> updateCityStat
  -> encodePathValue(originalKey.city)
  -> encodePathValue(originalKey.country)
  -> buildUrl("/:city/:country")
  -> fetch PUT /api/v2/citys-stats/:city/:country
    -> backend v2 app.put(`${BASE_API_URL}/${city}/${country}`)
    -> normalizeCountryForStorage(req.params.country)
    -> parseCityStat
      -> hasExactCityFields
      -> normalizeCountryForStorage(body.country)
      -> SUPPORTED_COUNTRIES
    -> comprueba URL y body
    -> db.findOne
    -> db.update
    -> db.findOne
    -> removeDatabaseId
    -> res.status(200).json(...)
  -> handleResponse(response)
-> actualiza form
-> message

```

Guardar cambiando `city` o `country` :

```

handleUpdate
-> clearFeedback
-> validateForm
  -> parsePositiveInteger
-> isSameResource false
-> createCityStat(payload)
  -> buildUrl()
  -> fetch POST /api/v2/citys-stats
  -> backend v2 app.post(BASE_API_URL)
    -> parseCityStat
      -> hasExactCityFields
      -> normalizeCountryForStorage
      -> SUPPORTED_COUNTRIES
    -> db.findOne
    -> db.insert
    -> removeDatabaseId
  -> handleResponse(response)
-> deleteCityStat(originalKey.city, originalKey.country)
  -> encodePathValue
  -> buildUrl("/:city/:country")
  -> fetch DELETE /api/v2/citys-stats/:city/:country
  -> backend v2 app.delete(`${BASE_API_URL}/${city}/${country}`)
    -> db.remove
  -> handleResponse(response)
-> updateRoute(created.city, created.country)
  -> encodeURIComponent
  -> replace("/citys-stats/editar/:city/:country")

```

Si estas en `/analytics/citys-stats`

```
onMount
-> loadAnalytics
-> getAllCityStats({ sort: "-un_2025_population" })
-> buildUrl("", { sort })
-> fetch GET /api/v2/citys-stats?sort=-un_2025_population
  -> backend v2 app.get(BASE_API_URL)
    -> db.find
    -> docs.map(removeDatabaseId)
    -> sort inline dentro de la ruta
    -> result.slice(...)
  -> handleResponse(response)
-> tick
-> loadHighcharts
  -> import("highcharts")
  -> import("highcharts/modules/accessibility.js")
-> renderChart
  -> citysStats.slice()
  -> sort por poblacion
  -> map(...)
    -> labelFor(item)
  -> chart?.destroy()
  -> Highcharts.chart(...)

onDestroy
-> chart?.destroy()
```

Si estas en `/analytics/citys-stats/map`

```
onMount
-> loadMapData
-> getAllCityStats({ sort: "-un_2025_population" })
-> buildUrl("", { sort })
-> fetch GET /api/v2/citys-stats?sort=-un_2025_population
-> backend v2 app.get(BASE_API_URL)
-> handleResponse(response)
-> declaraciones reactivas de Svelte:
-> geolocated = cityStats.map(...)
->   keyFor(item)
-> missing = cityStats.filter(...)
->   keyFor(item)
-> maxPopulation = Math.max(...)
-> minPopulation = Math.min(...)
-> points = geolocated.map(...)
->   colorFor(item.population)
->   radiusFor(item.population)
-> selectedPoint = points.find(...)
-> totalPopulation = geolocated.reduce(...)
-> tick
-> loadHighchartsMap
-> import("highcharts")
-> import("highcharts/modules/map.js")
-> import("highcharts/modules/accessibility.js")
-> renderMap
-> points.map(...)
->   titleCase(point.city)
->   titleCase(point.country)
-> Highcharts.mapChart(...)
-> evento render
->   removeCityMarkerClip(chart)
-> eventos click/mouseOver
->   selectPoint(point)

onDestroy
-> mapChart?.destroy()
```

Si estas en `/integrations/citys-stats`

La pantalla se puede lanzar de tres formas:

```
onMount(loadIntegrations)
select on:change={loadIntegrations}
button on:click={loadIntegrations}
```

Carga del resumen agregado:

```
loadIntegrations
-> destroyIntegrationCharts
-> getCityStatsIntegrationSummary(selectedLimit)
  -> fetch GET /api/v1/citys-stats/integrations/summary?limit=N
  -> backend v1 app.get(`${BASE_API_URL}/integrations/summary`)
    -> parseLimit
    -> findAllCityStats
      -> db.find
      -> docs.map(removeDatabaseId)
    -> buildCityCountrySummaries
      -> normalizeCountryKey
      -> readFiniteNumber
    -> Promise.all(countrySummaries.map(buildIntegratedCityBase))
      -> safeExternal(Open-Meteo)
      -> safeExternal(REST Countries)
    -> getWorldBankPopulations(countryCodes)
    -> Promise.all de APIs SOS externas
      -> getTouristArrivals
      -> getEarthquakes
      -> getFifaSquadValues
      -> getEsportsEarnings
    -> buildIntegratedCity
      -> devuelve items + studentApiDatasets
    -> handleResponse(response)
```

Si no hay datos locales:

```
if countrySummaries.length === 0
-> loadInitialCityStats
  -> buildUrl("/loadInitialData")
  -> fetch GET /api/v2/citys-stats/loadInitialData
  -> backend v2 app.get(`${BASE_API_URL}/loadInitialData`)
    -> db.count
    -> db.find o db.insert(initialData)
    -> removeDatabaseId
  -> handleResponse(response)
-> getCityStatsIntegrationSummary(selectedLimit) otra vez
```

Preparar datos finales de pantalla:

```
geocodingRows = countrySummaries.map(...)
countryCards = countrySummaries.map(... countryInfo ...)
worldBankRows = countrySummaries.map(... worldBankPopulation ...)
touristCountries = summaryDataset(summary, "touristCountries", "totalArrivals")
earthquakeCountries = summaryDataset(summary, "earthquakeCountries", "maxSeverity")
fifaCountries = summaryDataset(summary, "fifaCountries", "latestTotalMarketValue")
esportsCountries = summaryDataset(summary, "esportsCountries", "topCountryEarnings")
integrationErrors = collectSummaryErrors(summary, countrySummaries)
```

Punto importante para defensa:

```
El resumen NO se cachea.
Cada petición lee NeDB de nuevo.
Solo se cachean respuestas de APIs externas dentro de fetchJson.
```

Final de la pantalla:

```

loading = false
tick
loadHighcharts
  -> import("highcharts")
  -> import("highcharts/highcharts-more.js")
  -> highcharts-more registra columnpyramid y series avanzadas
  -> loadPlugin(dumbbell) como dependencia interna de lollipop
  -> loadPlugin(lollipop)
  -> loadPlugin(bullet)
  -> loadPlugin(sankey)
  -> loadPlugin(heatmap) como dependencia interna de treemap
  -> loadPlugin(treemap) para integracion 1 y como dependencia de sunburst
  -> loadPlugin(sunburst)
  -> loadPlugin(accessibility)
renderIntegrationCharts
  -> destroyIntegrationCharts
  -> renderGeocodingChart
    -> numberOrNull
    -> titleCase
    -> displayNumber/displayDecimal
    -> createChart
  -> renderCountryChart
    -> numberOrNull
    -> countryName
    -> displayNumber/displayCompact
    -> createChart
  -> renderWorldBankChart
    -> numberOrNull
    -> countryName
    -> displayNumber
    -> createChart
  -> renderTourismChart
    -> combinedCountryRows
      -> findLocalCountry
        -> localCountryIndex
          -> normalizeCountryKey
      -> localAveragePopulation si no hay match
    -> localPopulationFor
    -> titleCase
    -> createChart
  -> renderEarthquakeChart
    -> combinedCountryRows
    -> localPopulationFor
    -> normalizedIndex
      -> numberOrNull
    -> titleCase
    -> createChart
  -> renderFifaChart
    -> numberOrNull
    -> normalizedIndex
    -> titleCase
    -> createChart
  -> renderEsportsChart
    -> combinedCountryRows
    -> localPopulationFor
    -> normalizedIndex
    -> titleCase
    -> createChart
onDestroy
  -> destroyIntegrationCharts

```

Si estas en `/analytics`

```
onMount
-> loadAnalytics
-> Promise.all([
  getAllCityStats(),
  getDisasters(),
  getAllWineStats()
])
-> getAllCityStats
-> buildUrl
-> fetch GET /api/v2/city-stats
-> backend v2 app.get(BASE_API_URL)
-> handleResponse
-> getDisasters
-> fetch a la API de natural-disasters
-> getAllWineStats
-> fetch a la API de wine-stats
-> buildMetrics(cityStats, disasters, wines)
-> sum(cityStats, "un_2025_population")
-> sum(disasters, "death_count")
-> sum(wines, "unit")
-> calcula index normalizado
-> tick
-> loadHighcharts
-> import("highcharts")
-> import("highcharts/modules/accessibility.js")
-> renderChart
-> Highcharts.chart(...)
onDestroy
-> chart?.destroy()
```

Si llamas directamente a la API v2

GET `/api/v2/city-stats` :

```
app.get(BASE_API_URL)
-> db.find
-> docs.map(removeDatabaseId)
-> filtro city si viene
  No hay funcion auxiliar.
-> filtro country si viene
  No hay funcion auxiliar.
-> filtro un_2025_population si viene
  No hay funcion auxiliar.
-> busqueda q si viene
  No hay funcion auxiliar.
-> sort si viene
  No hay funcion auxiliar.
-> offset/limit si vienen
  No hay funcion auxiliar.
-> result.slice(offset, offset + limit)
-> res.status(200).json(...)
```

POST `/api/v2/city-stats` :

```
app.post(BASE_API_URL)
-> parseCityStat
-> hasExactCityFields
-> normalizeCountryForStorage
-> SUPPORTED_COUNTRIES
-> db.findOne duplicado
-> db.insert
-> removeDatabaseId
-> res.status(201).json(...)
```

PUT /api/v2/citys-stats/:city/:country:

```
app.put(`${BASE_API_URL}/${city}/${country}`)
  -> normalizeCountryForStorage(req.params.country)
  -> parseCityStat
    -> hasExactCityFields
    -> normalizeCountryForStorage(body.country)
    -> SUPPORTED_COUNTRIES
  -> comprobar URL/body
  -> db.findOne
  -> db.update
  -> db.findOne
  -> removeDatabaseId
  -> res.status(200).json(...)
```

DELETE /api/v2/citys-stats:

```
app.delete(BASE_API_URL)
  -> db.remove({}, { multi: true })
  -> res.sendStatus(204)
```

DELETE /api/v2/citys-stats/:city/:country:

```
app.delete(`${BASE_API_URL}/${city}/${country}`)
  -> normaliza req.params.city y req.params.country
  -> db.remove({ city, country }, {})
  -> res.sendStatus(204)
  -> o res.status(404).json(...) si no existia
```

Si llamas directamente a integraciones v1

Endpoints individuales:

```

GET /api/v1/citys-stats/integrations/geocoding/:city
-> getGeocoding
-> cleanSearchTerm
-> fetchJson Open-Meteo
-> fetch
-> response.text()
-> JSON.parse

GET /api/v1/citys-stats/integrations/country/:country
-> getCountryInfo
-> cleanSearchTerm
-> fetchJson REST Countries
-> fetch
-> response.text()
-> JSON.parse

GET /api/v1/citys-stats/integrations/world-bank/:countryCode
-> getWorldBankPopulation
-> worldBankPopulationCache.has
-> fetchJson World Bank si no esta en cache
-> normalizeWorldBankRow
-> worldBankPopulationCache.set

GET /api/v1/citys-stats/integrations/sos-tourist-arrivals
-> getTouristArrivals
-> fetchJson
-> asArray
-> normalizeTouristArrival
  -> readFiniteNumber
-> buildTouristArrivalsByCountry
-> normalizeCountryKey
-> sort por totalArrivals

GET /api/v1/citys-stats/integrations/sos-earthquakes
-> getEarthquakes
-> fetchJson
-> asArray
-> normalizeEarthquake
  -> countryFromIso3
  -> readFiniteNumber
-> buildEarthquakesByCountry
-> normalizeCountryKey
-> sort por maxSeverity

GET /api/v1/citys-stats/integrations/sos-fifa-squad-values
-> getFifaSquadValues
-> fetchJson
-> asArray
-> normalizeFifaSquadValue
  -> readFiniteNumber
-> buildFifaSquadValuesByCountry
-> normalizeCountryKey
-> sort por latestTotalMarketValue

GET /api/v1/citys-stats/integrations/sos-esports-earnings
-> getEsportsEarnings
-> fetchJson
-> asArray
-> normalizeEsportsEarning
  -> readFiniteNumber
-> buildEsportsEarningsByCountry
-> normalizeCountryKey
-> sort por topCountryEarnings

```

Resumen integrado:


```

GET /api/v1/citys-stats/integrations/summary
-> parseLimit
-> findAllCityStats
  -> db.find
  -> docs.map(removeDatabaseId)
-> buildCityCountrySummaries
  -> normalizeCountryKey
  -> readFiniteNumber
-> Promise.all(countrySummaries.map(buildIntegratedCityBase))
  -> buildIntegratedCityBase
    -> Promise.all
      -> safeExternal("Open-Meteo", () => getGeocoding)
        -> getGeocoding
          -> cleanSearchTerm
          -> fetchJson
      -> safeExternal("REST Countries", () => getCountryInfo)
        -> getCountryInfo
          -> cleanSearchTerm
          -> fetchJson
-> extrae countryCodes
-> getWorldBankPopulations
  -> usa worldBankPopulationCache
  -> fetchJson si faltan codigos
  -> normalizeWorldBankRow
-> Promise.all APIs SOS
  -> safeExternal(..., getTouristArrivals)
    -> fetchJson
    -> asArray
    -> normalizeTouristArrival
  -> safeExternal(..., getEarthquakes)
    -> fetchJson
    -> asArray
    -> normalizeEarthquake
  -> safeExternal(..., getFifaSquadValues)
    -> fetchJson
    -> asArray
    -> normalizeFifaSquadValue
  -> safeExternal(..., getEsportsEarnings)
    -> fetchJson
    -> asArray
    -> normalizeEsportsEarning
-> buildTouristArrivalsByCountry si hay datos
-> buildEarthquakesByCountry si hay datos
-> buildFifaSquadValuesByCountry si hay datos
-> buildEsportsEarningsByCountry si hay datos
-> integrationBases.map(buildIntegratedCity)
  -> normalizeCountryKey
  -> busca datos en Maps externos
  -> crea integrationErrors
-> res.status(200).json(...)

```

Frase corta para examen

- ! Si me dan una funcion, primero miro en que pantalla, boton o endpoint se usa. El archivo se carga de arriba abajo, pero la ejecucion real empieza cuando una ruta de Express, un `onMount` de Svelte o un evento de usuario llama a esa funcion. Si un paso dice "No hay funcion auxiliar", es codigo escrito directamente dentro de esa ruta.

18. Codigos HTTP y contrato REST

Codigo	Uso en este proyecto
200	Lectura correcta o actualizacion con respuesta
201	Recurso creado o datos iniciales insertados
204	Borrado correcto sin cuerpo, usado en eliminaciones correctas
400	Body incorrecto, pais no soportado, query invalida o URL/body no coinciden
404	Recurso concreto no encontrado
405	Metodo no permitido para esa URL
409	Duplicado o coleccion ya cargada segun recurso
500	Error interno de servidor o base de datos
502	Error controlado al consultar API externa

Frases utiles:

! El codigo HTTP forma parte del contrato de la API, no es un detalle decorativo.

! No usamos URLs tipo `/crearCiudad` porque el metodo HTTP ya expresa la accion.

! `POST` crea en la coleccion; `PUT` actualiza un recurso que ya existe.

Contrato real de `citys-stats v2`

Esta es la tabla que conviene saberse para defensa LCC. Es la API principal del CRUD.

Metodo	Ruta	Que hace	Respuestas esperadas
GET	<code>/api/v2/citys-stats/docs</code>	Redirige a documentacion Postman	302 redireccion
GET	<code>/api/v2/citys-stats/loadInitialData</code>	Carga datos iniciales si la base esta vacia	201 si inserta, 200 si ya habia datos, 500 si falla NeDB
GET	<code>/api/v2/citys-stats</code>	Lista coleccion con filtros, <code>q</code> , <code>sort</code> , <code>offset</code> , <code>limit</code>	200, 400 si query/sort/paginacion invalida, 500
GET	<code>/api/v2/citys-stats/:city/:country</code>	Devuelve un registro concreto	200, 404 si no existe, 500
POST	<code>/api/v2/citys-stats</code>	Crea un registro nuevo	201, 400 body invalido o pais no soportado, 409 duplicado, 500
POST	<code>/api/v2/citys-stats/:city/:country</code>	No permitido sobre recurso concreto	405
PUT	<code>/api/v2/citys-stats</code>	No permitido sobre coleccion completa	405
PUT	<code>/api/v2/citys-stats/:city/:country</code>	Actualiza un registro concreto	200, 400 body invalido o pais no soportado, 400 si URL/body no coinciden, 404, 500
DELETE	<code>/api/v2/citys-stats</code>	Borra toda la coleccion	204, 500
DELETE	<code>/api/v2/citys-stats/:city/:country</code>	Borra un registro concreto	204, 404, 500

Frase perfecta:

! En REST, la ruta identifica el recurso y el metodo identifica la accion. Por eso `GET /api/v2/citys-stats` lee la coleccion, `POST /api/v2/citys-stats` crea en la coleccion, `PUT /api/v2/citys-stats/:city/:country` actualiza un registro concreto y `DELETE` borra.

Orden interno del `GET /api/v2/citys-stats`

El handler de listado aplica las operaciones en este orden:

```
1. db.find({}) lee todos los documentos.
2. removeDatabaseId quita _id de NeDB.
3. Filtro exacto por city.
4. Filtro exacto por country.
5. Filtro exacto por un_2025_population.
6. Búsqueda libre q en city o country.
7. sort por city, country o un_2025_population.
8. offset y limit para paginacion.
9. res.status(200).json(...)
```

Por que ese orden:

! Primero se obtiene una coleccion limpia, despues se reduce con filtros y busqueda, luego se ordena y al final se pagina. Si paginara antes de filtrar, podria perder resultados correctos.

Ejemplos:

```
GET /api/v2/citys-stats
GET /api/v2/citys-stats?country=china
GET /api/v2/citys-stats?q=india
GET /api/v2/citys-stats?sort=-un_2025_population&limit=5
GET /api/v2/citys-stats?offset=5&limit=5
```

Diferencia entre `POST` y `PUT`

`POST` :

```
POST /api/v2/citys-stats
```

Se usa sobre la coleccion porque el recurso todavia no existe. El backend decide si se puede crear.

Orden:

```
1. parseCityStat(req.body)
2. Si body invalido o pais no soportado -> 400
3. db.findOne busca duplicado por city + country
4. Si ya existe -> 409
5. db.insert crea
6. Devuelve 201 con el recurso creado
```

`PUT` :

```
PUT /api/v2/citys-stats/:city/:country
```

Se usa sobre un recurso concreto porque el recurso ya existe o deberia existir.

Orden:

1. Lee city y country de req.params.
2. parseCityStat(req.body).
3. Si body invalido o pais no soportado -> 400.
4. Si body.city/body.country no coinciden con URL -> 400.
5. db.findOne comprueba si existe.
6. Si no existe -> 404.
7. db.update actualiza.
8. db.findOne vuelve a leer actualizado.
9. Devuelve 200 con el recurso actualizado.

Frase perfecta:

- ! `POST` no necesita identificador en la URL porque esta creando dentro de la coleccion. `PUT` si lo necesita porque modifica un recurso concreto y por eso compruebo que URL y body coincidan.

Codigos que mas preguntan

400 Bad Request :

- ! El cliente ha mandado mal la peticion: body con estructura incorrecta, pais no soportado, query invalida, sort no permitido, offset/limit invalidos o URL/body que no coinciden.

404 Not Found :

- ! La ruta existe, pero el recurso concreto no esta en la base de datos.

405 Method Not Allowed :

- ! La URL existe pero ese metodo no tiene sentido ahi. Por ejemplo, `PUT /api/v2/citys-stats` no se permite porque `PUT` debe ir contra un recurso concreto.

409 Conflict :

- ! La peticion esta bien formada, pero choca con el estado actual. En `POST`, ocurre si ya existe la misma combinacion `city + country`.

500 Internal Server Error :

- ! Error interno de servidor o base de datos. No es culpa del cliente.

502 Bad Gateway :

- ! Se usa en integraciones cuando nuestro backend actua como proxy y falla una API externa.

Contrato de integraciones LCC

Las integraciones estan bajo:

```
/api/v1/citys-stats/integrations/...
```

Endpoints principales:

Ruta	Fuente	Codigos habituales
<code>/integrations/geocoding/:city</code>	Open-Meteo	200, 404, 502
<code>/integrations/country/:country</code>	REST Countries	200, 404, 502
<code>/integrations/world-bank/:countryCode</code>	World Bank	200, 404, 502
<code>/integrations/sos-tourist-arrivals</code>	API SOS externa	200, 502
<code>/integrations/sos-earthquakes</code>	API SOS externa	200, 502
<code>/integrations/sos-fifa-squad-values</code>	API SOS externa	200, 502
<code>/integrations/sos-esports-earnings</code>	API SOS externa	200, 502
<code>/integrations/summary?limit=5</code>	Resumen combinado	200, 400, 500

Frase de defensa:

! En integraciones mi backend no es la fuente original de todos los datos, sino un proxy controlador. Por eso si una API externa falla, respondo con errores controlados como 502 o con errores parciales dentro del resumen.

19. Cambios que pueden pedir en directo

Cambiar puerto

Archivo:

```
index.js
```

Pasos:

1. Buscar `const port = process.env.PORT || 10000`.
2. Cambiar `10000` si se pide otro puerto local.
3. Arrancar con `npm.cmd start`.
4. Ver consola.

Frase:

! En despliegue manda `process.env.PORT`; el `10000` es el valor por defecto local.

Cambiar donde se guarda una base

Archivo:

```
index.js
```

Buscar:

- `citysStatsDb`

Cambiar:

```
filename: path.join(__dirname, "src", "back", "citys-stats.db")
```

Cambiar datos iniciales

Archivos LCC:

```
src/back/v1/citys-stats.js
src/back/v2/citys-stats.js
```

Pasos generales:

1. Cambiar `initialData`.
2. Respetar estructura exacta.
3. Borrar coleccion.
4. Cargar datos iniciales.
5. Ejecutar test del recurso.

Ciudades:

```
Invoke-RestMethod -Method Delete http://localhost:10000/api/v2/citys-stats
Invoke-RestMethod http://localhost:10000/api/v2/citys-stats/loadInitialData
```

Anadir campo a `citys-stats`

Tocar:

```
src/back/v1/citys-stats.js
src/back/v2/citys-stats.js
frontend-group/src/routes/citys-stats/+page.svelte
frontend-group/src/routes/citys-stats/editar/[city]/[country]/+page.svelte
tests/LCC/pruebas-lcc*.json
tests/LCC/e2e/citys-stats.spec.js
```

Pasos:

1. Anadir campo en `hasExactCityFields`.
2. Limpiar y validar en `parseCityStat`.
3. Anadirlo a `initialData`.
4. Anadir input en formulario de crear.
5. Anadir input en formulario de editar.
6. Anadir columna en tabla.
7. Revisar analytics si usa el campo.
8. Actualizar tests.

Frase:

! Como la API valida estructura exacta, no basta con anadir un input. Hay que aceptar el campo en backend, normalizarlo, mostrarlo y probarlo.

Renombrar campo

Ejemplo:

```
un_2025_population -> population_2025
```

Pasos:

1. Ejecutar `rg "un_2025_population"`.
2. Cambiar backend v1 y v2.
3. Cambiar `initialData`.
4. Cambiar services si construyen payloads.
5. Cambiar formularios y tablas.
6. Cambiar analytics, mapa e integraciones.
7. Cambiar tests.
8. Ejecutar `rg "un_2025_population"` otra vez.

Anadir filtro

Ciudades:

1. Abrir `src/back/v2/citys-stats.js`.
2. Tocar `app.get(BASE_API_URL, ...)`.
3. Validar query param.
4. Abrir `frontend-group/src/routes/citys-stats/+page.svelte`.
5. Anadir campo a `emptySearchForm`.
6. Anadir input.
7. Anadirlo en `buildSearchQuery`.
8. Probar URL.

Anadir ordenacion

Ciudades ya tiene patron:

```
?sort=campo
?sort=-campo
```

Archivo:

```
src/back/v2/citys-stats.js
```

Buscar:

```
allowedFields
```

Anadir el campo permitido. No dejar ordenacion por campos arbitrarios.

Crear una ruta nueva de API

Pasos:

1. Elegir recurso y version.
2. Abrir archivo de API.
3. Poner ruta concreta antes de rutas parametrizadas.
4. Usar metodo correcto (`GET`, `POST`, `PUT`, `DELETE`).
5. Quitar `_id` si devuelve documentos.
6. Crear service si la usa frontend.
7. Pintarla si hace falta.
8. Crear o adaptar tests.

Ejemplo:

```
app.get(`${BASE_API_URL}/count`, (req, res) => {
  db.count({}, (err, count) => {
    if (err) return res.sendStatus(500);
    return res.status(200).json({ count });
  });
});
```

Importante:

! En Express importa el orden: `/count` debe ir antes de `/:city/:country`.

Crear v3

Pasos:

1. Crear `src/back/v3`.
2. Copiar como base v2 del recurso.
3. Cambiar `BASE_API_URL` a `/api/v3/...`.
4. Implementar diferencias.
5. Importar en `index.js`.
6. Registrar con `app` y la base correcta.
7. Crear tests v3.
8. Actualizar enlaces si procede.

Cambiar una pantalla o ruta frontend

Rutas actuales se crean por carpetas:

```
frontend-group/src/routes/nueva-ruta/+page.svelte
```

Si hay parametros:

```
frontend-group/src/routes/recurso/editar/[id]/+page.svelte
```

No hace falta registrar manualmente en `App.svelte`, porque se detecta con `import.meta.glob`.

Si quieres enlace visible:

```
frontend-group/src/components/Navbar.svelte
```

Cambiar menu

Archivo:

```
frontend-group/src/components/Navbar.svelte
```

Los enlaces actuales son rutas directas:

```
/
/citys-stats
/analytics
/integrations
```

Cambiar portada

Archivo:

```
frontend-group/src/routes/+page.svelte
```

Cuidado:

- Playwright LCC revisa `data-testid` de la tarjeta LCC.
- No cambiar `member-citys-stats`, `frontend-citys-stats`, `api-v1-citys-stats`, etc. sin actualizar tests.

Cambiar grafica Highcharts

Archivos:

```
frontend-group/src/routes/analytics/+page.svelte
frontend-group/src/routes/analytics/citys-stats/+page.svelte
frontend-group/src/routes/analytics/citys-stats/map/+page.svelte
frontend-group/src/routes/integrations/citys-stats/+page.svelte
```

Patron:

1. Cargar datos.
2. Transformar datos.
3. Cargar Highcharts.
4. Renderizar.
5. Destruir grafica en `onDestroy`.

Anadir ciudad al mapa

Archivo:

```
frontend-group/src/routes/analytics/citys-stats/map/+page.svelte
```

Objeto:

```
coordinates
```

Clave:

```
city|country
```

Ejemplo:

```
"madrid|spain": { lat: 40.4168, lon: -3.7038 }
```

20. Errores comunes y soluciones

npm.ps1 no se puede cargar

Causa:

PowerShell bloquea scripts.

Solucion:

```
npm.cmd install
npm.cmd run build
npm.cmd start
```

npm start falla

Comprobar:

- Estas en la raíz del proyecto.
- Ejecutaste `npm install`.
- El puerto `10000` no esta ocupado.
- No hay error de sintaxis en el ultimo cambio.

La web se ve antigua

Causa:

`npm start` sirve `public`, no el codigo fuente de `frontend-group/src`.

Solucion:

```
npm.cmd run build
npm.cmd start
```

Puerto ocupado

Soluciones:

- Cerrar proceso que usa `10000`.
- Cambiar `PORT`.
- Cambiar temporalmente el valor por defecto en `index.js`.

loadInitialData dice que ya hay datos

loadInitialData solo inserta datos si la coleccion esta vacia. Si ya existen registros, en LCC devuelve los datos actuales o informa de que no se insertan duplicados.

Solucion rapida para reiniciar ciudades:

```
Invoke-RestMethod -Method Delete http://localhost:10000/api/v2/citys-stats
Invoke-RestMethod http://localhost:10000/api/v2/citys-stats/loadInitialData
```

API devuelve 400

Puede ser:

- Body incompleto.
- Campo extra.
- Numero no valido.
- Query invalida.
- URL y body no coinciden en `PUT`.

Mirar:

- `normalize...`
- `hasExact...`
- `buildSearchQuery`

API devuelve 409

Significa conflicto.

Ejemplos:

- Ciudad duplicada por `city + country`.

API devuelve 404

Significa recurso no encontrado.

Comprobar:

- Identificador correcto.
- Texto normalizado.
- Recurso no borrado previamente.

API devuelve 405

Significa metodo no permitido.

Ejemplos:

```
POST /api/v2/citys-stats/tokyo/japan
PUT /api/v2/citys-stats
```

Frase:

! No es un fallo, es parte del contrato REST.

Integraciones fallan

Puede pasar por:

- API externa dormida.
- Timeout.
- Error en formato JSON externo.
- Problema temporal de red.

Mirar:

- `fetchJson`
- `safeExternal`
- `integrationErrors`

Frase:

! Si una fuente externa falla, el backend conserva el resto y comunica el fallo como error parcial.

21. Decisiones tecnicas

Express como servidor unico

Decision:

Usar un unico servidor Express para API y frontend compilado.

Ventaja:

- Facil despliegue en Render.
- Misma origin para frontend y API en produccion.
- Fallback sencillo para rutas SPA.

NeDB como persistencia

Decision:

Usar NeDB con un archivo `.db` por recurso.

Ventaja:

- Sencillo para la practica.
- No requiere servidor de base de datos externo.
- Documentos JSON faciles de entender.

Limitacion:

- No es una base preparada para alta concurrencia o produccion real a gran escala.

Versionado v1/v2

Decision:

Mantener v1 y v2 conviviendo.

Ventaja:

- No se rompe el contrato anterior.
- Se pueden anadir mejoras.
- El frontend puede elegir la version que necesita.

Validacion estricta

Decision:

Rechazar campos extra o cuerpos incompletos.

Ventaja:

- Contrato claro.
- Tests mas predecibles.
- Evita datos basura.

Proxy propio para integraciones

Decision:

El navegador no llama directamente a APIs externas en LCC; llama a nuestro backend.

Ventaja:

- Evita problemas de Same Origin Policy.
- Centraliza timeout y errores.
- Normaliza formatos antes de pintar.

Rutas por carpetas en frontend

Decision:

Usar `+page.svelte` y `import.meta.glob` para descubrir pantallas.

Ventaja:

- Anadir ruta suele ser crear carpeta y archivo.
- Las rutas directas funcionan con fallback Express.

22. Limitaciones y mejoras futuras

Limitaciones actuales:

- NeDB es local y sencillo, no una base empresarial.
- Las integraciones externas dependen de servicios de terceros y Render puede estar dormido.
- Algunas APIs externas pueden cambiar formato.
- Algunas partes de otros recursos tienen estilos y tests menos homogéneos que LCC.
- No hay autenticación de usuarios.
- No hay migraciones de esquema para cambios grandes.

Mejoras futuras:

- Migrar persistencia a PostgreSQL, MongoDB o similar.
- Anadir tests e2e homogéneos para todos los recursos.
- Anadir cache persistente para integraciones externas.
- Anadir observabilidad: logs estructurados y métricas.
- Mejorar validación con un schema formal como JSON Schema o Zod.
- Crear una API v3 si se necesitan cambios incompatibles.
- Unificar criterios visuales entre todas las pantallas.

Comprobaciones externas no verificables desde el código:

- Videos personales, si la entrega los exige.
- Informes Toggl/efforts, si la entrega los exige.
- PR formal asociado a issue done y milestone/release, si la entrega lo pide.

Estos puntos no se pueden confirmar mirando solo el repositorio local, así que no deben presentarse como errores del proyecto ni como funcionalidades pendientes del código.

23. Guion de defensa practica

Esta seccion esta reescrita para la defensa real: no es un discurso cerrado. Es un guion de actuacion para cuando el profesor te pida tareas concretas delante del sistema, Chrome DevTools, Postman y el codigo.

Regla de oro:

pantalla correcta -> accion real -> Network/Postman -> codigo -> prediccion exacta -> diagrama por capas

Frase base si te bloqueas:

! Voy a seguir el flujo real: el usuario actua en el HTML, Svelte ejecuta una funcion, el service hace `fetch`, Express recibe la peticion, valida los datos, consulta o modifica NeDB, y devuelve JSON para que el frontend actualice tabla, grafica o mapa.

No hagas esto:

- No expliques el proyecto en abstracto si te han pedido una tarea concreta.
- No cambies codigo si te han pedido usar la interfaz.
- No edites la base de datos manualmente.
- No pulses `Send` en Postman si te dicen que solo predigas.
- No digas "dara error"; di codigo HTTP y respuesta exacta.

Preparacion antes de entrar

Tener abiertas estas pestanas:

1. `https://sos2526-29.onrender.com/`
2. `https://sos2526-29.onrender.com/citys-stats`
3. `https://sos2526-29.onrender.com/analytics/citys-stats`
4. `https://sos2526-29.onrender.com/analytics/citys-stats/map`
5. `https://sos2526-29.onrender.com/integrations/citys-stats`
6. `https://sos2526-29.onrender.com/analytics`

Tener abiertos estos archivos:

1. `index.js`
2. `src/back/v2/citys-stats.js`
3. `src/back/v1/citys-stats.js`
4. `frontend-group/src/services/apiBase.js`
5. `frontend-group/src/services/citysStatsApi.js`
6. `frontend-group/src/routes/citys-stats/+page.svelte`
7. `frontend-group/src/routes/citys-stats/editar/[city]/[country]/+page.svelte`
8. `frontend-group/src/routes/analytics/citys-stats/+page.svelte`
9. `frontend-group/src/routes/analytics/citys-stats/map/+page.svelte`
10. `frontend-group/src/routes/integrations/citys-stats/+page.svelte`

Base URL para Postman:

`https://sos2526-29.onrender.com`

Endpoint principal LCC:

`/api/v2/citys-stats`

Tarea 1: abrir el sistema desplegado

Si dicen:

Abre tu sistema desplegado en la nube.
Abre tu aplicacion desplegada en Render.
Ensename tu sistema funcionando en produccion.

Abres:

<https://sos2526-29.onrender.com/>

Frase:

! Este es el sistema completo desplegado en Render. Express sirve el frontend compilado desde `public` y tambien expone las rutas `/api`.

Si Render tarda:

! Render puede dormir la aplicacion si lleva tiempo sin uso. Mientras arranca puedo enseñar el código y después volver a la pestaña.

Tarea 2: abrir la interfaz de gestion de datos

Si dicen:

Entra en la interfaz de gestion de tus datos.
Abre la parte donde se muestran los datos.
Abre la grafica donde aparecen tus datos.
Ensename la pantalla donde puedes gestionar tus datos.

Si piden gestionar, crear, editar o borrar, abres:

`/citys-stats`

Si piden literalmente la grafica, abres:

`/analytics/citys-stats`

Y dices:

! La grafica no se edita directamente. La uso para identificar el dato; para cambiarlo voy a `/citys-stats`, edito el registro y luego vuelvo a la grafica para comprobar que se ha actualizado.

Frase:

! Esta es la interfaz de gestion de mi recurso `citys-stats`. Desde aqui puedo cargar datos iniciales, buscar, crear, editar y borrar registros usando la UI que llama a mi API REST.

Senala:

- Boton `Cargar datos de ejemplo`.
- Formulario de busqueda.
- Formulario de creacion.
- Tabla.
- Botones `Editar` y `Eliminar`.

Archivos relacionados:

```
frontend-group/src/routes/citys-stats/+page.svelte
frontend-group/src/services/citysStatsApi.js
src/back/v2/citys-stats.js
```

Tarea 3: modificar un dato desde la interfaz

Si dicen:

Cambia este valor usando tu interfaz.
Modifica este dato desde tu aplicacion.
Haz que en la grafica aparezca este nuevo valor.

Haces:

1. Ir a `/citys-stats`.
2. Si no hay datos, pulsar `Cargar datos de ejemplo`.
3. Buscar el registro por ciudad o pais.
4. Pulsar `Editar`.
5. Cambiar `Poblacion estimada en 2025`.
6. Pulsar `Guardar cambios`.
7. Volver a `/citys-stats` si hace falta.
8. Abrir `/analytics/citys-stats` para comprobar la grafica.

Frase mientras lo haces:

! Lo cambio desde la interfaz, no desde el codigo ni desde la base de datos. La pantalla de edicion llama al service y el service hace una peticion `PUT` al backend.

Donde tocas literalmente:

```
/citys-stats -> boton Editar -> campo "Poblacion estimada en 2025" -> Guardar cambios
```

Flujo si solo cambia la poblacion:

```
HTML submit
-> handleUpdate()
-> updateCityStat()
-> PUT /api/v2/citys-stats/:city/:country con JSON
-> parseCityStat()
-> db.findOne()
-> db.update()
-> db.findOne()
-> 200 con JSON actualizado
```

Si cambia `city` o `country`:

```
createCityStat(nuevo)
-> POST /api/v2/citys-stats
-> deleteCityStat(antiguo)
-> DELETE /api/v2/citys-stats/:city/:country
```

Frase:

- ! Como la clave es `city + country`, si cambia la clave no es un `PUT` normal sobre el mismo recurso: se crea el nuevo y se borra el anterior.

Tarea 4: comprobar el cambio en la grafica

Abres:

```
/analytics/citys-stats
```

Frase:

- ! Esta grafica no tiene datos escritos a mano. Al abrirse ejecuta `loadAnalytics`, llama a `getAllCitysStats({ sort: "-un_2025_population" })` y pinta Highcharts con lo que devuelve la API.

Peticion esperada en Network:

```
GET /api/v2/citys-stats?sort=-un_2025_population
```

Archivo:

```
frontend-group/src/routes/analytics/citys-stats/+page.svelte
```

Tarea 5: abrir DevTools y localizar la peticion

Abrir DevTools:

```
F12  
Ctrl + Shift + I
```

Pasos:

1. Ir a `Network`.
2. Filtrar por `Fetch/XHR`.
3. Recargar la pagina.
4. Elegir la peticion que empieza por `/api`.

Peticiones que debes reconocer:

Accion	Peticion
Listado	GET /api/v2/citys-stats
Busqueda	GET /api/v2/citys-stats?q=...&sort=...
Crear	POST /api/v2/citys-stats
Abrir edicion	GET /api/v2/citys-stats/:city/:country
Guardar edicion	PUT /api/v2/citys-stats/:city/:country
Borrar	DELETE /api/v2/citys-stats/:city/:country
Grafica	GET /api/v2/citys-stats?sort=-un_2025_population
Mapa	GET /api/v2/citys-stats?sort=-un_2025_population
Integraciones	GET /api/v1/citys-stats/integrations/summary?limit=N

Frase:

! En **Headers** se ve el metodo, la URL y el status. En **Payload** se ve el JSON enviado en **POST** o **PUT**. En **Response** o **Preview** se ve el JSON que devuelve la API.

Tarea 5B: que mirar exactamente en Network

Esta parte es importante porque el profesor puede abrir DevTools y pedir:

Senalame cual es la peticion que carga los datos.
 Senalame si esta peticion pasa por proxy.
 Explicame por que salen tantas lineas en Network.

Primero, no te asustes por ver muchas filas. Network mezcla cosas distintas:

Tipo en Network	Que es	Importancia
document	La pagina HTML que abre el navegador, por ejemplo /citys-stats	No es la API de datos
script	JS compilado de Svelte/Vite, por ejemplo index-....js	No es la API de datos
stylesheet	CSS compilado, por ejemplo index-....css	No es la API de datos
png , svg	Imagenes, favicon o banderas	No es la API de datos
fetch	Peticiones hechas con fetch() desde el frontend	Estas son las importantes

Filtro recomendado:

Network -> Fetch/XHR

Si aun asi ves muchas cosas, mira la columna **Name** y abre solo las filas que empiecen por:

/api/

O haz clic en la fila y mira en **Headers** el campo:

Request URL

Chrome a veces muestra en **Name** solo la ultima parte de la URL. Por ejemplo puede mostrar **citys-stats**, pero al abrirla en **Headers** se ve la ruta completa:

```
/api/v2/citys-stats
```

Caso 1: entrar en /citys-stats

Captura real esperada al abrir la pantalla:

```
GET 200 document /citys-stats
GET 200 stylesheet /assets/index-....css
GET 200 script /assets/chunk-....js
GET 200 script /assets/index-....js
GET 200 fetch /api/v2/citys-stats
```

La petición de datos es:

```
GET /api/v2/citys-stats
```

Respuesta:

```
200 OK
Array JSON con los registros de citys-stats
```

Frase para decir:

! La fila importante es la de tipo `fetch` hacia `/api/v2/citys-stats`. Las filas `script`, `stylesheet` o `document` son solo recursos de la web. Esta petición no es un proxy externo: es el frontend llamando a mi backend REST en el mismo dominio.

Caso 2: crear un registro desde /citys-stats

Al crear desde el formulario deben aparecer dos peticiones importantes:

```
POST 201 fetch /api/v2/citys-stats
GET 200 fetch /api/v2/citys-stats
```

Explicación:

1. `POST /api/v2/citys-stats`: envía el JSON nuevo al backend.
2. El backend valida, comprueba duplicados e inserta en NeDB.
3. Si va bien devuelve `201 Created`.
4. Después el frontend recarga la lista con `GET /api/v2/citys-stats`.

En la fila del `POST`, abre:

```
Payload
```

Debe verse un JSON parecido:

```
{
  "city": "granada",
  "country": "spain",
  "un_2025_population": 230000
}
```

En `Response` o `Preview` debe verse el objeto creado.

Si ya existe la ciudad y el país:

```
POST /api/v2/citys-stats -> 409 Conflict
```

Si el país no existe o no está soportado:

```
POST /api/v2/citys-stats -> 400 Bad Request
Response: { "error": "Invalid country" }
```

Ojo con la diferencia entre interfaz y Postman:

- Desde la interfaz normal, el frontend valida primero con `isSupportedCountry`, así que lo habitual es que no llegue a aparecer el `POST`.
- Si alguien se salta la interfaz y manda el dato con Postman, la API lo rechaza igualmente con `400`.

Frase para decir:

- ! Al crear, la acción real de escritura es el `POST`. El `GET` posterior es la recarga automática de la tabla para que el usuario vea el dato nuevo sin refrescar a mano.

Frase si te preguntan por el error de World Bank:

- ! Ese error aparecía porque había un país local que no se podía traducir a ISO3. Ahora no dejo guardar países no soportados: el frontend lo avisa antes de enviar y el backend lo bloquea con `400 Invalid country` si llega por Postman.

Caso 3: buscar con filtros

Ejemplo real:

```
GET /api/v2/citys-stats?country=china&sort=-un_2025_population&limit=1
```

Respuesta:

```
200 OK
Array JSON filtrado, ordenado y limitado
```

Frase:

- ! Los filtros no se aplican solo en el navegador. El frontend construye query params y la API responde ya con el subconjunto pedido.

Caso 4: abrir y guardar edición

Al abrir una edición:

```
GET /api/v2/citys-stats/:city/:country
```

Ejemplo:

```
GET /api/v2/citys-stats/tokyo/japan
```

Al guardar sin cambiar `city` ni `country`:

```
PUT /api/v2/citys-stats/tokyo/japan
```

En `Payload` se ve el JSON enviado. En `Response` se ve el objeto actualizado.

Si cambias `city` o `country`, como la clave del recurso es compuesta, la interfaz hace:

```
POST /api/v2/citys-stats
DELETE /api/v2/citys-stats/:oldCity/:oldCountry
```

Frase:

- ! Si solo cambia la poblacion es un `PUT`. Si cambia la clave `city + country`, no puedo actualizar el identificador directamente; creo el recurso nuevo y borro el antiguo.

Caso 5: entrar en `/integrations/citys-stats`

Captura real actual esperada:

```
GET 200 document /integrations/citys-stats
GET 304 script /assets/index-....js
GET 304 css /assets/index-....css
GET 200 fetch /api/v1/citys-stats/integrations/summary?limit=8
GET 200 script /assets/highcharts-more-....js
GET 200 script /assets/sankey-....js
GET 200 script /assets/treemap-....js
GET 200 script /assets/sunburst-....js
```

La peticion importante es:

```
GET /api/v1/citys-stats/integrations/summary?limit=8
```

Respuesta:

```
200 OK
JSON resumen con datos locales, APIs no SOS y APIs SOS
```

Frase:

- ! En integraciones el navegador no llama directamente a Open-Meteo, REST Countries, World Bank ni a las APIs SOS externas. El navegador hace una unica llamada a mi backend: `/api/v1/citys-stats/integrations/summary?limit=8`. Luego Express, en el servidor, hace las peticiones externas, normaliza los datos y devuelve un JSON preparado para los widgets.

Esto es proxy propio porque:

```
navegador -> mi Express en Render -> APIs externas
```

No es:

```
navegador -> APIs externas directamente
```

Que significa 304

304 Not Modified no es un fallo. Significa:

El navegador pregunto si ese recurso habia cambiado.
El servidor respondio que no.
Chrome reutiliza la version cacheada.

Suele salir en:

```
document
script
stylesheet
imagenes
```

Tambien puede aparecer en alguna llamada si Chrome esta reutilizando cache. Para ver todo como nuevo:

DevTools abierto -> Network -> marcar Disable cache -> recargar

Frase:

! 304 no significa error de mi API. Significa que el navegador ha usado cache porque el recurso no ha cambiado. Si quiero ver la respuesta completa, marco Disable cache y recargo con DevTools abierto.

Si aparecen muchas peticiones antiguas en integraciones

Antes, la vista podia lanzar muchas llamadas separadas, por ejemplo:

```
IND
CHN
IDN
BGD
sos-tourist-arrivals
sos-earthquakes
sos-fifa-squad-values
sos-esports-earnings
```

Eso era justo el problema de "demasiadas peticiones". La solucion actual es agruparlo en:

```
GET /api/v1/citys-stats/integrations/summary?limit=8
```

Si en Render todavia ves muchas filas antiguas:

1. Puede que Render aun no haya terminado el deploy.
2. Puede que Chrome tenga JS antiguo cacheado.
3. Haz hard reload:

Ctrl + Shift + R

O:

DevTools -> Network -> Disable cache -> recargar

Frase:

! Si veo muchas llamadas antiguas es porque estoy usando un build viejo o cacheado. En la version corregida, la pantalla de integraciones carga mediante una sola llamada summary al backend.

Tarea 6: explicar si usas proxy

Respuesta correcta, separando casos:

Caso	Que decir
CRUD en Render	No pasa por proxy de Vite; frontend y backend comparten dominio.
CRUD en desarrollo	<code>apiBase.js</code> usa <code>http://localhost:10000</code> en modo dev.
Proxy de Vite	Existe en <code>vite.config.js</code> para llamadas relativas <code>/api</code> .
Integraciones	Si hay proxy propio: el navegador llama a Express y Express llama a APIs externas.

Archivos:

```
frontend-group/src/services/apiBase.js
frontend-group/vite.config.js
src/back/v1/citys-stats.js
```

Frase para demostrarlo:

! En Network, para CRUD veo llamadas a `/api/v2/citys-stats` en el mismo origen de Render. Para integraciones veo una llamada a `/api/v1/citys-stats/integrations/summary?limit=N`; despues, en el backend, Express hace el `fetch` a APIs externas.

Tarea 7: Postman

Base URL:

```
https://sos2526-29.onrender.com
```

Si lo haces en local:

```
http://localhost:10000
```

Header para `POST` y `PUT`:

```
Content-Type: application/json
```

Body valido:

```
{
  "city": "defensa-demo",
  "country": "spain",
  "un_2025_population": 123456
}
```

Peticiones:

Accion	Metodo y endpoint	Respuesta esperada
Listar	GET /api/v2/citys-stats	200 y array JSON
Cargar iniciales	GET /api/v2/citys-stats/loadInitialData	201 si estaba vacia, 200 si ya habia datos
Crear	POST /api/v2/citys-stats	201 y objeto creado
Leer uno	GET /api/v2/citys-stats/defensa-demo/spain	200 u 404
Actualizar	PUT /api/v2/citys-stats/defensa-demo/spain	200 y objeto actualizado
Borrar uno	DELETE /api/v2/citys-stats/defensa-demo/spain	204 u 404
Borrar todo	DELETE /api/v2/citys-stats	204

Si piden PATCH :

 Mi CRUD no implementa PATCH . Implementa GET , POST , PUT y DELETE . Si haces PATCH /api/v2/citys-stats/defensa-demo/spain , Express no encontrara una ruta PATCH y respondera 404 con la respuesta por defecto, no con un JSON propio de mi API.

Para que el POST sea predecible, usa una ciudad que no exista o borra antes defensa-demo . Si ya existe, el resultado correcto no es 201 , sino 409 .

Tarea 8: predecir respuesta sin pulsar Send

Tabla que debes memorizar:

Peticion	Condicion	Codig o	Respuesta
GET /api/v2/citys-stats	DB accesible	200	Array JSON sin _id
GET /api/v2/citys-stats?un_2025_population=abc	Valor no numerico	400	{ "error": "Invalid query" }
GET /api/v2/citys-stats?sort=bad	Campo no permitido	400	{ "error": "Invalid sort field" }
GET /api/v2/citys-stats?offset=-1	Offset invalido	400	{ "error": "Invalid offset" }
GET /api/v2/citys-stats?limit=-1	Limit invalido	400	{ "error": "Invalid limit" }
GET /api/v2/citys-stats/nope/spain	No existe	404	{ "error": "Resource not found" }
POST /api/v2/citys-stats	Body valido y no duplicado	201	Objeto creado
POST /api/v2/citys-stats	Duplicado	409	{ "error": "Resource already exists" }
POST /api/v2/citys-stats	Body mal estructurado	400	{ "error": "JSON body does not match expected structure" }
POST /api/v2/citys-stats	Pais no soportado	400	{ "error": "Invalid country" }
POST /api/v2/citys-stats/:city:country	POST sobre recurso concreto	405	Sin JSON propio
PUT /api/v2/citys-stats	PUT sobre coleccion	405	Sin JSON propio
PUT /api/v2/citys-stats/tokyo/japan	Pais del body no soportado	400	{ "error": "Invalid country" }
PUT /api/v2/citys-stats/tokyo/japan	Body no coincide con URL	400	{ "error": "URL and body do not match" }
PUT /api/v2/citys-stats/nope/spain	No existe	404	{ "error": "Resource not found" }
DELETE /api/v2/citys-stats/tokyo/japan	Existe	204	Sin cuerpo
DELETE /api/v2/citys-stats/nope/spain	No existe	404	{ "error": "Resource not found" }
PATCH /api/v2/citys-stats/tokyo/japan	Metodo no implementado	404	Respuesta por defecto de Express

Campos exactos validos:

```
city
country
un_2025_population
```

La poblacion debe ser entero mayor que cero.

El pais debe existir en la lista soportada. Si el JSON tiene estructura correcta pero `country` no se puede cruzar con las integraciones, la respuesta esperada es:

```
400
{ "error": "Invalid country" }
```

Si preguntan por un campo `year`:

! En mi recurso LCC `citys-stats` no existe campo `year`. Mis campos son `city`, `country` y `un_2025_population`. La validacion equivalente a "valor invalido" es que `un_2025_population` no sea entero mayor que cero.

Si el JSON esta roto de sintaxis, por ejemplo falta una llave, puede fallar antes de entrar al endpoint, en `express.json()`. Eso tambien es `400`, pero no lo genera `parseCityStat`.

Frase para escribir:

!Codigo de estado: `400`. Respuesta: `{ "error": "JSON body does not match expected structure" }`, porque el body no tiene exactamente `city`, `country` y `un_2025_population` con poblacion entera mayor que cero.

Tarea 9: explicar frontend

Si dicen "abre el frontend", abre segun la pregunta:

Pregunta	Archivo
Donde se decide el backend	<code>frontend-group/src/services/apiBase.js</code>
Donde estan los fetch CRUD	<code>frontend-group/src/services/citysStatsApi.js</code>
Donde esta el listado	<code>frontend-group/src/routes/citys-stats/+page.svelte</code>
Donde esta editar	<code>frontend-group/src/routes/citys-stats/editar/[city]/[country]/+page.svelte</code>
Donde esta la grafica	<code>frontend-group/src/routes/analytics/citys-stats/+page.svelte</code>
Donde esta el mapa	<code>frontend-group/src/routes/analytics/citys-stats/map/+page.svelte</code>
Donde estan integraciones	<code>frontend-group/src/services/citysStatsIntegrations.js</code>

Respuestas cortas:

Pregunta	Respuesta
Que es <code>item</code> , <code>d</code> o <code>row</code>	Un elemento del array que se esta recorriendo o transformando.
Por que <code>map</code>	Transforma cada registro en otro formato.
Por que <code>filter</code>	Deja solo los elementos que cumplen una condicion.
Por que <code>find</code>	Busca un elemento concreto.
Por que <code>await</code>	Espera una promesa antes de usar el resultado.
Que hace <code>tick()</code>	Espera a que Svelte haya pintado el contenedor HTML antes de crear Highcharts.

Plantilla:

! Esta linea pertenece al frontend. No toca NeDB directamente. Prepara datos, llama a un service, espera una promesa o actualiza variables que Svelte usa para repintar la vista.

Tarea 10: explicar backend y validaciones

Archivo principal:

```
src/back/v2/citys-stats.js
```

Integraciones:

```
src/back/v1/citys-stats.js
```

Funciones clave:

Funcion	Explicacion
<code>removeDatabaseId</code>	Quita <code>_id</code> de NeDB antes de responder.
<code>hasExactCityFields</code>	Comprueba que el body tenga exactamente los campos esperados.
<code>normalizeCountryForStorage</code>	Limpia pais, quita tildes, aplica guiones y resuelve alias.
<code>parseCityStat</code>	Valida estructura, poblacion y pais soportado; devuelve <code>item</code> o <code>error</code> .
<code>normalizeCityStat</code>	Envoltorio compatible que delega en <code>parseCityStat</code> .
<code>db.find</code>	Lista registros.
<code>db.findOne</code>	Busca uno o comprueba duplicados.
<code>db.insert</code>	Crea.
<code>db.update</code>	Actualiza.
<code>db.remove</code>	Borra.

Explicar `POST`:

1. Entra en `app.post(BASE_API_URL)`.
2. Ejecuta `parseCityStat(req.body)`.
3. Si falla estructura, poblacion o pais soportado, responde 400.
4. Busca duplicado con `db.findOne`.
5. Si existe, responde 409.
6. Si no existe, inserta con `db.insert`.
7. Quita `_id` y responde 201.

Explicar `PUT`:

1. Lee `city` y `country` de `req.params`.
2. Normaliza `country` de la URL.
3. Valida el body con `parseCityStat`.
4. Comprueba que URL y body coinciden.
5. Busca si el recurso existe.
6. Si no existe, responde 404.
7. Actualiza con `db.update`.
8. Vuelve a buscar y responde 200.

Si falla NeDB:

! En las rutas CRUD, si la base de datos devuelve error, el backend responde `500`. Ese codigo significa error interno del servidor, no error de la peticion del cliente.

Tarea 11: orden de ejecucion y asincronia

Arranque del servidor:

1. Node lee index.js de arriba abajo.
2. Importa Express, path, NeDB y cors.
3. Crea app y puerto.
4. Registra cors y express.json.
5. Crea las bases NeDB.
6. Importa los modulos de API.
7. Cada modulo registra rutas.
8. Registra proxies y frontend estatico.
9. Ejecuta app.listen.
10. Las rutas solo se ejecutan cuando llega una peticion.

Frase:

- ! Declarar funciones y registrar rutas no ejecuta la logica de una peticion. El handler se ejecuta cuando llega un metodo y una URL que coinciden.

Orden de un `POST` :

```
handler POST
-> parseCityStat
-> si falla, 400
-> db.findOne
-> si existe, 409
-> db.insert
-> removeDatabaseId
-> 201 con JSON
```

Si hay `await` :

- ! La funcion se pausa hasta que se resuelve la promesa. Node no se queda bloqueado para siempre; cuando llega el resultado continua justo despues del `await` o entra en `catch` si falla.

Tarea 11B: ejercicio de letras A, B, C

Si el profesor da numeros de linea y dice:

Imagina que al principio de estas lineas hay `console.log A, B, C...`
Predice que secuencia saldria al arrancar el servidor y hacer `loadInitialData`.

Hazlo asi:

1. No ejecutes nada al principio; primero predice en papel.
2. Marca las lineas con comentarios en la misma linea, por ejemplo `/* A */`.
3. No anadas lineas nuevas, porque cambiarian los numeros.
4. Separa dos fases: arranque del servidor y peticion HTTP.
5. En el arranque, Node ejecuta `index.js` de arriba abajo y registra rutas, pero no ejecuta handlers.
6. Cuando llega `GET /api/v2/citys-stats/loadInitialData`, entonces entra en el handler de esa ruta.
7. Las llamadas a NeDB son asincronas con callback; el codigo que esta dentro del callback sale despues de que NeDB responda.
8. Si te deja comprobarlo despues, cambia los comentarios por `console.log("A")`, ejecuta y compara.

Frase clave:

! Que una ruta este escrita en el archivo no significa que se ejecute al arrancar. Al arrancar solo se registra. El handler de `loadInitialData` se ejecuta cuando llega esa petición desde Postman o desde la interfaz.

Tarea 11C: chuleta para no fallar el orden de ejecucion

Cuando te den lineas concretas, clasificalas antes de intentar decir letras:

Tipo de linea	Se ejecuta al arrancar el servidor	Se ejecuta al llegar una petición	Como explicarlo
<code>const express = require(...)</code>	Si	No	Node carga dependencias al leer <code>index.js</code> .
<code>const app = express()</code>	Si	No	Se crea la aplicacion Express al arrancar.
<code>app.use(cors())</code>	Si, se registra	Despues actua como middleware en cada petición	Al arrancar se instala; en peticiones se aplica.
<code>const api = require("../src/back/v2/citys-stats")</code>	Si	No	Carga el modulo de la API.
<code>citysStatsApiV2(app, db)</code>	Si	No	Ejecuta el modulo para registrar rutas.
<code>function parseCityStat(...)</code>	No por si sola	Solo si alguien la llama	Declarar una funcion no ejecuta su cuerpo.
<code>app.get("/ruta", (req, res) => { ... })</code>	Si, registra la ruta	El cuerpo se ejecuta si llega esa ruta	La cabecera registra; el interior espera petición.
Codigo dentro del callback de <code>app.get</code>	No	Si coincide metodo y URL	Es el handler real de la petición.
<code>db.count(..., callback)</code>	Dentro de la petición	Si	Lanza una operacion asincrona a NeDB.
Codigo dentro del callback de NeDB	No inmediatamente	Despues de que NeDB responda	Sale mas tarde que el codigo sincrono que ya estaba en marcha.
<code>return res.status(...).json(...)</code>	No al arrancar	Si esa rama se alcanza	Envia la respuesta y termina esa rama.

Regla practica:

1. Primero apunta las letras que estan en codigo de arranque.
2. Ignora el cuerpo de handlers hasta que exista una petición.
3. Cuando haya petición, entra solo en la ruta que coincide con metodo + URL.
4. Dentro de esa ruta, sigue el codigo de arriba abajo.
5. Si aparece NeDB o fetch con await, marca que ahi hay espera asincrona.
6. El callback o lo que va despues del await continua cuando llega el resultado.

Ejemplo mental para `GET /api/v2/citys-stats/loadInitialData` desde servidor recién arrancado:

```

FASE 1: arranque
index.js importa librerias
index.js crea app y bases NeDB
index.js carga citys-stats v2
citys-stats v2 declara constantes y funciones
citys-stats v2 registra app.get("/loadInitialData")
index.js registra frontend y listen

FASE 2: peticion
Postman o frontend pide GET /api/v2/citys-stats/loadInitialData
Express busca la ruta que coincide
entra en el handler de loadInitialData
llama a db.count({})
cuando NeDB responde:
  si count > 0:
    llama a db.find({})
    cuando NeDB responde:
      responde 200 con array sin _id
  si count === 0:
    llama a db.insert(initialData)
    cuando NeDB responde:
      responde 201 con array sin _id

```

Frase para defender asincronia:

! En JavaScript el archivo se lee de arriba abajo, pero una peticion HTTP y una consulta a NeDB no bloquean todo el programa. El handler arranca, llama a NeDB, y el codigo del callback continua cuando NeDB devuelve el resultado. Por eso, en el ejercicio de letras, las letras dentro de callbacks pueden salir mas tarde que las lineas sincronas.

Si te pierdes durante el ejercicio:

1. Di en voz baja: "arranque o peticion".
2. Si es arranque, solo cuentan imports, constantes, creacion de app, bases, llamadas a modulos, registro de rutas y `listen`.
3. Si es peticion, solo cuenta la ruta exacta que coincide.
4. Si ves `db.find`, `db.count`, `db.insert`, `db.update` o `db.remove`, entra al callback despues.
5. Si ves `return`, esa rama ya no sigue bajando.

Tarea 12: marcar lineas sin cambiar numeros

Si te dicen "marca esta linea con A sin cambiar la numeracion", no anadas una linea encima.

Correcto:

```
const parsed = parseCityStat(req.body); // A
```

Incorrecto:

```
// A
const parsed = parseCityStat(req.body);
```

Tarea 13: GitHub, local y Render sincronizados

Antes de defender:

```
git status --short
```

Frase:

! GitHub es la referencia entregada. Local debe coincidir para que los numeros de linea sean iguales, y Render debe estar desplegado desde esa version.

Tarea 14: diagrama de secuencia

Capas correctas para LCC:

```
html -> Svelte script -> express -> NeDB
```

No decir MongoDB: este proyecto usa NeDB.

Crear dato:

```
html -> Svelte script:
  submit del formulario

Svelte script -> express:
  POST /api/v2/citys-stats con JSON

express -> NeDB:
  db.findOne para comprobar duplicado

NeDB -> express:
  doc o null

express -> NeDB:
  db.insert si no existe

NeDB -> express:
  newDoc con _id interno

express -> Svelte script:
  201 con JSON creado sin _id

Svelte script -> express:
  GET /api/v2/citys-stats para refrescar

express -> Svelte script:
  200 con array JSON

Svelte script -> html:
  tabla actualizada
```

Donde viaja JSON:

- En el **POST** viaja JSON hacia Express.
- En la respuesta **201** vuelve JSON creado.
- En el **GET** de refresco no hay body, pero la respuesta trae array JSON.
- Entre Express y NeDB no hay HTTP; es llamada interna.

Editar dato:

```

html -> Svelte script:
  submit de guardar cambios

Svelte script -> express:
  PUT /api/v2/citys-stats/:city/:country con JSON

express -> NeDB:
  db.findOne para comprobar existencia

express -> NeDB:
  db.update para actualizar

express -> NeDB:
  db.findOne para leer el actualizado

express -> Svelte script:
  200 con JSON actualizado

Svelte script -> html:
  mensaje de exito y formulario actualizado

```

Borrar dato:

```

html -> Svelte script:
  click en Eliminar

Svelte script -> express:
  DELETE /api/v2/citys-stats/:city/:country sin body

express -> NeDB:
  db.remove

NeDB -> express:
  numRemoved

express -> Svelte script:
  204 sin cuerpo si se borro
  404 con JSON si no existia

Svelte script -> express:
  GET /api/v2/citys-stats para refrescar

express -> Svelte script:
  200 con array JSON

```

Cargar grafica:

```

html -> Svelte script:
  se abre /analytics/citys-stats

Svelte script -> express:
  GET /api/v2/citys-stats?sort=-un_2025_population

express -> NeDB:
  db.find({})

NeDB -> express:
  documentos

express -> Svelte script:
  200 con array JSON

Svelte script -> html:
  Highcharts pinta la grafica

```

Respuestas relampago

Orden o pregunta	Respuesta
Abre el sistema desplegado	<code>https://sos2526-29.onrender.com/</code>
Abre gestion de datos	<code>/citys-stats</code>
Abre la grafica	<code>/analytics/citys-stats</code>
Que peticion carga datos	GET <code>/api/v2/citys-stats</code>
Que metodo crea	POST
Que metodo edita	PUT
Que metodo borra	DELETE
Que pasa si falta un campo	400 con <code>JSON body does not match expected structure</code>
Que pasa si ya existe	409 con <code>Resource already exists</code>
Que pasa si el pais no existe	400 con <code>Invalid country</code>
Que pasa si no existe	404 con <code>Resource not found</code>
Que base usas	NeDB
Donde se valida	<code>parseCityStat</code> , <code>hasExactCityFields</code> y <code>SUPPORTED_COUNTRIES</code>
Donde se ve el JSON	Network: <code>Payload</code> , <code>Response</code> , <code>Preview</code>

Simulacro practico cronometrado

1. Abrir Render.
2. Abrir `/citys-stats`.
3. Cargar datos iniciales si hace falta.
4. Crear `defensa-demo` , `spain` , `123456` .
5. Abrir DevTools y localizar el `POST` .
6. Explicar metodo, URL, payload, status y respuesta.
7. Editar poblacion a `654321` .
8. Localizar el `PUT` .
9. Abrir `/analytics/citys-stats` .
10. Localizar el `GET` de la grafica.
11. Abrir Postman.
12. Hacer `GET` , `POST` , `PUT` , `DELETE` .
13. Preparar una peticion mala sin pulsar `Send` .
14. Predecir codigo y JSON.
15. Dibujar `html -> Svelte script -> express -> NeDB` .

24. Preguntas difíciles y respuestas

Por que REST y no rutas tipo `/crearCiudad`

Porque REST identifica recursos con URLs y usa metodos HTTP para las acciones. La URL dice "que recurso" y el metodo dice "que accion".

Por que `POST` crea en la coleccion

Porque antes de crear no existe la URL del recurso concreto. La coleccion es donde nace el nuevo recurso.

Por que `PUT` lleva identificador en URL

Porque actualiza un recurso existente. Ademas se comprueba que URL y body coincidan para evitar modificar otro registro por error.

Por que se usan query params

Porque filtros, busqueda, orden y paginacion no cambian el recurso base; solo cambian la vista que se devuelve.

Por que NeDB

Porque para esta practica necesitamos persistencia sencilla en archivos. NeDB permite documentos JSON sin instalar una base externa. Cada recurso tiene su propio `.db`.

Por que se quita `_id`

Porque `_id` es interno de NeDB. La API publica no debe depender de detalles de almacenamiento.

Por que hay v1 y v2

Porque v1 mantiene compatibilidad y v2 anade mejoras sin romper clientes previos. En LCC, v2 se usa para CRUD y v1 aloja integraciones.

Por que algunas rutas usan v1 en frontend

Porque `citysStatsIntegrations.js` llama a integraciones que estan implementadas en `/api/v1/citys-stats/integrations/...`.

Por que `citys-stats` esta escrito asi

Porque es el contrato publico usado por rutas, tests y documentacion. Cambiarlo romperia enlaces y entregables.

Por que claves compuestas

Porque no todos los datasets necesitan un `id` numerico. En LCC, una ciudad se identifica de forma natural con `city + country`.

Que pasa si mando un campo de mas

La API responde `400`, porque la validacion exige estructura exacta.

Por que `DELETE` devuelve a veces `204`

Porque el borrado fue correcto y no hace falta devolver cuerpo.

Que hace CORS

Permite que el frontend en desarrollo, que puede estar en otro puerto, llame al backend sin bloqueo del navegador.

Como se despliega el frontend

Vite compila `frontend-group/src` hacia `public`. Express sirve `public` con `express.static`.

Por que funcionan rutas directas como `/analytics`

Porque Express devuelve `index.html` para rutas que no son `/api`, y Svelte resuelve la pantalla por `window.location.pathname`.

Donde esta el proxy propio

En `src/back/v1/citys-stats.js`, bajo `/api/v1/citys-stats/integrations/...`.

Que pasa si una API externa falla

`fetchJson` y `safeExternal` controlan el error. La vista puede mostrar datos parciales e informar avisos.

Por que no se muestra JSON crudo

Porque JSON es el formato de intercambio. En la interfaz se transforma en widgets, tablas, tarjetas y metricas.

Como se relacionan datos locales y externos

Primero se agregan ciudades por pais. Despues se normalizan nombres de pais e ISO3 para cruzarlos con las fuentes externas.

Donde se ve el trabajo de grupo

En los tres recursos, la portada, las APIs, tests, `/analytics` y `/integrations`.

25. Glosario para personas no tecnicas

Termino	Explicacion
Backend	Parte del sistema que recibe peticiones, valida datos y habla con la base
Frontend	Parte visible que usa el usuario en el navegador
API	Forma ordenada de pedir o enviar datos a un servidor
REST	Estilo de API basado en recursos, URLs y metodos HTTP
Recurso	Tipo de dato gestionado, por ejemplo <code>citys-stats</code>
CRUD	Crear, leer, actualizar y borrar
JSON	Formato de texto para intercambiar datos
NeDB	Base de datos local que guarda documentos en archivos
Endpoint	Ruta concreta de una API
Query param	Parametro de URL como <code>?limit=5</code>
Proxy	Servidor intermedio que llama a otra API por nosotros
SPA	Aplicacion web que carga una vez y cambia pantallas con JavaScript
Build	Version compilada y lista para servir
Test e2e	Prueba que simula acciones reales en navegador
Newman	Herramienta para ejecutar colecciones Postman
Highcharts	Libreria para graficas y mapas

26. Checklists finales

Antes de defender

```
npm.cmd install
npm.cmd --prefix frontend-group install
npm.cmd run build
npm.cmd start
```

Abrir:

- `http://localhost:10000/`
- `http://localhost:10000/citys-stats`
- `http://localhost:10000/analytics`
- `http://localhost:10000/analytics/citys-stats`
- `http://localhost:10000/analytics/citys-stats/map`
- `http://localhost:10000/integrations`
- `http://localhost:10000/integrations/citys-stats`
- `http://localhost:10000/api/v2/citys-stats`
- `http://localhost:10000/api/v1/citys-stats/integrations/summary?limit=8`

Tests utiles

```
npm.cmd run test-LCC-v2
npm.cmd run test-LCC-e2e
```

Despues de tocar backend

- Reiniciar `npm start`.
- Probar endpoint directo.
- Ejecutar Newman del recurso.

Despues de tocar frontend

- Ejecutar `npm.cmd run build` si se va a probar con `npm start`.
- Abrir ruta afectada.
- Revisar consola del navegador.
- Ejecutar Playwright si afecta a flujo visible.

Despues de tocar campos

- Ejecutar `rg "nombre_del_campo"`.
- Actualizar backend, frontend, analytics, integraciones y tests.
- Probar crear y editar.

Checklist LCC D03

- CRUD v2 accesible.
- Documentacion Postman v2 enlazada.
- Carga inicial disponible.
- Busqueda libre `q`.
- Filtros exactos.
- Ordenacion `sort`.
- Paginacion `limit` y `offset`.
- Grafico individual no `line`.
- Mapa geoespacial.
- Integraciones por pais.
- 7 APIs integradas.
- 3 APIs no SOS.
- 4 APIs SOS externas.
- Proxy propio.
- Vista `/integrations`.
- Widget grupal unico en `/analytics`.

27. Anexos tecnicos

Comandos API rapidos para `citys-stats`

```
Invoke-RestMethod http://localhost:10000/api/v2/citys-stats
Invoke-RestMethod http://localhost:10000/api/v2/citys-stats/loadInitialData
Invoke-RestMethod "http://localhost:10000/api/v2/citys-stats?q=india"
Invoke-RestMethod "http://localhost:10000/api/v2/citys-stats?sort=-un_2025_population&limit=5"
```

Crear ciudad:

```
Invoke-RestMethod -Method Post `
  -Uri http://localhost:10000/api/v2/citys-stats `
  -ContentType "application/json" `
  -Body '{"city":"madrid","country":"spain","un_2025_population":7000000}'
```

Borrar todo:

```
Invoke-RestMethod -Method Delete http://localhost:10000/api/v2/citys-stats
```

Rutas de documentacion Postman

```
/api/v1/citys-stats/docs
/api/v2/citys-stats/docs
```

Regla de oro para cambios

Si cambia una API:

```
backend -> service -> pantalla -> tests -> build
```

Si cambia solo una grafica:

```
pantalla analytics/integrations -> renderChart/renderMap -> probar ruta
```

Si cambia un campo:

```
backend + initialData + frontend + analytics + tests
```

Estado final de `docs`

Los documentos antiguos de defensa dentro de `docs/` ya no deben usarse para estudiar, preparar cambios ni defender el proyecto. Se eliminan como fuentes independientes para evitar duplicar informacion y para que nadie tenga que buscar explicaciones repartidas en varios sitios.

Las unicas referencias activas dentro de `docs/` deben ser:

```
docs/DEFENSA_COMPLETA.md
```

El PDF antiguo puede usarse solo como copia derivada si se regenera desde este Markdown.